

**LOG ANALYSIS AND FAILURE PREDICTION FOR DATA
PIPELINE BATCH JOBS USING XGBOOST ALGORITHM**

BY

AJIBOLA, MOJOLAOLUWA DAVID

22CG031820

**A PROJECT PROPOSAL SUBMITTED TO THE
DEPARTMENT OF COMPUTER AND INFORMATION
SCIENCES, COLLEGE OF SCIENCE AND TECHNOLOGY,
COVENANT UNIVERSITY OTA, OGUN STATE.**

**IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE BACHELOR OF SCIENCE
(HONOURS) DEGREE IN COMPUTER SCIENCE**

JUNE 2026

CERTIFICATION

I hereby certify that this project was carried out by Ajibola, Mojolaoluwa David in the department of Computer and Information Sciences, college of Science and Technology, Covenant University, Ota, Ogun State, Nigeria under my supervision.

Dr Iheanetu Olamma

Supervisor

Signature and Date

Prof. Oni Aderonke

Head of Department

Signature and Date

ACKNOWLEDGEMENTS

My most profound gratitude goes to Almighty God, who has kept me and sustained me from the very start of my degree until now. To Him be all the glory.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS	ii
TABLE OF CONTENTS	iii
LIST OF FIGURES	v
LIST OF FIGURES	vi
ABBREVIATIONS	vii
CHAPTER ONE	1
INTRODUCTION	1
1.1. Background Information	1
1.2. Statement of the Problem	3
1.3 Aim and Objectives of the Study	3
1.4 Methodology	3
Table 1.1: Objectives-Methodology Table	4
1.5 Significance of the study	5
1.6 Scope of the study	5
1.7 Project Outline	6
CHAPTER TWO	7
LITERATURE REVIEW	7
2.1 Preamble	7
2.2 Review of Data Engineering	7
2.2.1 Evolution of ETL Architecture	7
2.2.2 Components and Architecture of ETL pipelines	8
2.3.1 Impacts and costs of Failed Batch Jobs	10
2.3.2 Categories of Extract Load Transform Batch Job Failures	10
2.4 Review of Machine Learning Algorithms for Batch Job Failure Prediction	11
2.4.1 Review of Log Analysis using Machine Learning	13
2.5 Review of Existing Models and Gaps	14

2.5.1	Analysis of Job Failure and Prediction Model for Cloud Computing using Machine Learning	14
2.5.2	DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning	15
2.5.3	Cloud failure prediction based on traditional machine learning and deep learning	16
2.6	Gaps Identified in Literature	16
REFERENCES		17

LIST OF FIGURES

FIGURES	TITLES OF FIGURES	PAGES
2.1	A simple representation of an ETL pipeline	9
2.2	Machine Learning Techniques	17
2.3	Workflow for Log Analysis and Prediction	31
2.4	DeepLog Architecture	32

LIST OF FIGURES

FIGURES	TITLES OF FIGURES	PAGES
1.1	Objectives Methodology Table	9

ABBREVIATIONS

AI - Artificial Intelligence

API - Application Programming Interface

CPU- Central Processing Unit

DT- Decision Trees

ELT- Extract Load Transform

ETL – Extract Transform Load

GB – Gradient Boost

HPC- High Performance Computing

I/O – Input /Output

IOT- Internet of Things

IT – Information Technology

KNN- K-Nearest Neighbor

LSTM – Long Short-Term Memory

ML – Machine Learning

NB- Naïve Bayes

NoSQL – Not Only Structured Query Language

QDA – Quadratic Discriminant Analysis

RNN- Recurrent Neural Network

CHAPTER ONE

INTRODUCTION

1.1. Background Information

In the modern data-oriented environment, the role of organizations is becoming dependent on data pipelines to Extract, Transform, and Load (ETL) large volumes of information on various sources into central repositories to analyze it and make decisions. Data pipelines play a crucial role in today's data-centric age and have become fundamental components in enterprise Information Technology (IT) infrastructures. By collecting, processing, and transferring data, data pipelines make it possible to use the ever-growing amounts of information to gain valuable insights and make improved decisions ([Foidl et al., 2024](#)). Modern data engineering thrives on data pipelines, which are automated data collection, data movement, and data transformations processes performed on incoming data to bring them to specific destinations. Enterprise data architecture is based on these pipelines, which are the basis of business intelligence, predictive analytics, and operational reporting ([Foidl et al., 2024](#)).

Although classic data pipeline designs remain useful in many batch-oriented environments, they are under increasing pressure to support high-volume, high-velocity, and diverse data streams. Modern pipelines often combine structured and semi-structured sources, changing schemas, cloud storage, scheduled transformations, and downstream analytics dependencies. These conditions make manual monitoring less reliable because a small ingestion, cleaning, or transformation defect can move silently into other stages of the pipeline before the issue becomes visible to users ([Chanda, 2024](#)).

Recent studies show that data-related pipeline problems often emerge around type mismatches, cleaning, ingestion, integration, and compatibility issues ([Foidl et al., 2024](#)). These findings support the need for automated monitoring and prediction because a failure can affect multiple downstream systems before it is discovered by a data engineer. The present study therefore treats machine learning as a support mechanism for early warning, not as a replacement for engineering judgement or sound pipeline design.

Artificial intelligence and machine learning can support pipeline management in areas

such as anomaly detection, error prioritization, schema monitoring, resource awareness, and operational decision support. In this report, the role of machine learning is narrowed to log analysis and batch-job failure prediction so that the implementation remains measurable and defensible. The project does not claim full autonomous pipeline repair; it demonstrates how predictive evidence can be presented to an operator before or during a risky execution state ([Seenivasan, 2025](#)).

Machine learning-based log analysis and failure prediction in data pipelines is therefore an important direction in data engineering. Conventional monitoring is often reactive because errors are handled after failure messages, missing outputs, or downstream quality issues appear. A predictive monitoring layer can instead summarize historical and live execution evidence, detect patterns associated with risk, and support proactive intervention before a batch job causes wider disruption ([Popovic et al., 2024](#)). Automated ETL research also supports the need for stronger data quality and governance controls within pipeline workflows ([Ogunsola et al., 2022](#)).

The goal of this study is to apply XGBoost to the data and ETL pipeline systems to predict the possible mistakes that could be generated by the batch jobs. This study will attempt to determine the root cause or causes of why the batch error occurred and how the error can be rectified before occurrence in future situations.

1.2. Statement of the Problem

The non-observability of data pipeline batch job failures in enterprise settings makes them unnoticed until these failures result in production data quality problems. Manual log analysis makes data engineering teams spend majority of resources on data pipeline failures in the shape of reactive maintenance efforts instead of proactive pipeline optimization ([Akash & Charate, 2025](#)).

These failure patterns are not automated in the existing monitoring systems; consequently, organizations spend hours per end-to-end data integration task, causing cascading failures that affect subsequent analytics, reporting, and business decision-making processes.

1.3 Aim and Objectives of the Study

The aim of this study is to design and develop a log analysis and failure prediction system for ETL batch jobs using XGBoost. The specific objectives include:

- i. to identify and characterize common failure patterns in data pipeline batch jobs.
- ii. to carry out a comparative analysis based on accuracy metrics for multiple machine learning algorithms to effectively predict pipeline failures.
- iii. to develop a prediction model to continuously monitor pipeline execution of batch jobs.
- iv. to test and evaluate the system using the following metrics, Precision, Recall and F1-Score.

1.4 Methodology

The development of a log analysis and failure prediction application for data pipeline batch jobs follows a systematic, iterative approach integrating data science, machine learning engineering, and software development methodologies.

The study implements a secondary data collection method, whereby quantitative data relating to the study is gathered from online datasets and data repositories. The study utilizes data relating to pipeline execution logs, system performance metrics and historical failure records to identify and characterize batch job failures. The study will comparatively compare multiple machine learning algorithms for their effectiveness in predicting pipeline failures.

The study uses an agile methodology approach to develop the failure prediction application in small iterations that can continuously monitor pipeline execution for batch jobs. The study will finally evaluate the system using Precision, Recall and F1-Score metrics.

Table 1.1: Objectives-Methodology Table

S/N	OBJECTIVES	METHODOLOGY
1.	To collect data and prepare the machine learning algorithm for the prevention of pipeline batch job failures	Identify and collect all relevant data sources including pipeline execution logs, system performance metrics and historical failure records.
2.	To evaluate and compare multiple machine learning algorithms for their effectiveness in predicting pipeline failures.	Identify algorithms suitable for nature of pipeline failure prediction.
3.	To implement a prediction system that can continuously monitor pipeline execution.	To develop the proposed model as a microservice or Application Programming Interface (API) that can receive pipeline data and return failure predictions.
4.	To test and evaluate the system using the following metrics using the metrics, Precision, Recall and F1-Score.	Compute the confusion matrix consisting of True Positives, True Negatives, False Positives and False Negatives and comparing scores with benchmark metrics.

1.5 Significance of the study

This study is relevant across multiple fields of study. It suggests techniques for detecting and predicting errors in data pipelines before they happen. It also provides comprehensive information of the core factors and reasons why a particular batch job failed. Firstly, this research contributes to the growing body of knowledge at the intersection of machine learning and data engineering, an area that has received increasing attention as organizations grapple with the challenges of managing complex data ecosystems. While significant research has focused on optimizing individual components of data pipelines such as transformation efficiency, storage optimization, or query performance relatively less attention has been devoted to holistic, intelligent pipeline management systems capable of learning from operational history and predicting failures ([Foidl et al., 2024](#)).

Secondly, this study is underscored by the operational cost of pipeline failure and the burden of maintaining complex data ecosystems. Data-related incidents can affect reporting accuracy, compliance activities, customer-facing services, and downstream analytics ([Foidl et al., 2024](#)). The value of the proposed system is therefore not only in model accuracy, but also in the way it converts logs, metrics, and alerts into an interpretable monitoring workflow that helps engineers respond earlier ([Ogunsola et al., 2022](#)).

Lastly, the study also contributes to the growing discourse on AI-driven infrastructure management and autonomous systems, demonstrating how machine learning can be applied not just to business analytics but to the fundamental infrastructure that enables those analytics. As organizations continue to grapple with increasing data complexity and velocity, the principles and techniques developed in this research will remain relevant and can be extended to address emerging challenges in real-time streaming pipelines, edge computing scenarios, and distributed data processing environments.

1.6 Scope of the study

This study focuses on developing a machine learning-driven framework for predicting failures in batch-oriented data pipeline jobs, specifically ETL pipelines. The research covers log-based data sources including application logs, system logs, and database logs to identify and characterize failure patterns within data pipelines.

The study encompasses multiple categories of pipeline failures including resource exhaustion (memory, Central Processing Unit (CPU), disk space), data quality anomalies (schema mismatches, null values, data type errors), and dependency-related disruptions (upstream failures, connection timeouts). The research evaluates supervised learning models and ensemble techniques as the primary methodological approaches for failure prediction.

The scope includes the complete end-to-end predictive pipeline covering log data ingestion and it operates within environments where sufficient historical log data is available, logging practices are consistent across pipeline components, and there exists meaningful frequency and diversity of failure events to support model development and evaluation.

1.7 Project Outline

This study will consist of five comprehensive chapters that will adequately explain the research question. Chapter one consists of the introduction to the research, background information, statement of problem, scope and significance of the study and a little intro into the methodology of the research. Chapter 2 will consist of a very comprehensive literature review of various papers related to the study. It will talk about core concepts of the study and look at already existing systems and their limitations; it will also look at the gaps this research aims to cover.

Chapter three will consist of the methodology; it will include an extensive and deep dive into the specific method and algorithms that the study is implementing to achieve the aims. It will talk about the various datasets the study will utilize Chapter four will comprise of the results and analysis the study was able to achieve, it will show whether the study was successful or not in achieving the goal. Chapter five will consist of the project conclusion and areas where future research can be conducted or improved upon.

CHAPTER TWO

LITERATUR REVIEW

2.1 Preamble

This chapter reviews available literature, and systems relating to failure prediction, log analysis and machine learning methods. It talks about data engineering as a field of study, ETL pipelines and architecture and the evolution of ETL architecture. This chapter also discusses batch jobs, batch job failures, their impacts and costs, machine learning and all its traditional algorithms. It also reviews the gaps in already existing systems and the specific area this study covers.

2.2 Review of Data Engineering

Data engineering is a core discipline in modern data-driven organizations. It focuses on designing and building that allows data to be extracted and processed. This processed data can then be used for analytics, for the training of machine learning models or to make business informed decisions. Data pipelines are the core technology behind this process and they could be in the form of Extract, Transform, Load (ETL) or Extract, Load, Transform (ELT) pipelines.

ETL pipelines are the architectural basis of integration and processing of data in the present-day data-driven companies. These pipelines are the core of data warehousing and business intelligence systems as they enable movement and transformation of data that could have various origins and load to the target destinations where it could be analyzed and be used in a decision-making process.

2.2.1 Evolution of ETL Architecture

Legacy ETL pipelines were typified by a batch processing paradigm rule-based transformations and the necessity that they can be configured manually. They relied heavily on manual interventions which introduced the potential for human error during data extraction, transformation or loading([Ogunsola et al., 2022](#)). Such legacy systems were not particularly good at managing schema changes, and had to be maintained

manually with significant effort, and have not been very flexible to handle real-time data processing.

Modern ETL is much more advanced than the earliest iterations with use of cloud-native platforms real time data integration, which replaces on-premises processing in batch mode. Automated ETL is aimed at reducing the level of manual interferences when designing, managing and enhancing data pipelines ([Chanda, 2024](#)). Various factors have contributed to this transformation; the volumes of data are increasing exponentially, real time analytics are required, there is also the spread of a variety of data sources and there is the implementation of cloud computing infrastructure.

2.2.2 Components and Architecture of ETL pipelines

ETL pipelines are built on three basic steps with each carrying a specific role in the process of data integration. The extraction stage entails the mining of such data in various heterogeneous forms of relational databases, Not Only Structured Query Language (NoSQL) stores as well as Application Programming Interfaces (API's), Internet of Things (IOT) devices, flat files, and streaming platforms. This stage supports different data formats, connection protocol, as well as access methods and guarantees data integrity when they are being accessed. The process typically occurs during scheduled intervals when system load is minimal, with the main goal being to gather all necessary data from sources efficiently ([Nishanth R. M., 2019](#)).

The transformation phase is the most complicated stage in ETL process because it involves cleaning of the data, validation, normalization, enriching and aggregation of data and applying business logic. Some of the challenges that transformations must conquer are data quality problems like missing values, duplications, inconsistency as well as format inconsistency. Contemporary transformation engines today often include checks on data quality, schema evolution management and inference of data type or types to minimize human activity. This process may involve data deduplication, format standardization, and applying business rules to make the data uniform and suitable for analysis ([Nishanth R. M., 2019](#)).

The loading step sends processed data to the target systems, which can be data warehouses, data lakes, machine-learning feature stores, or operational databases. Strategies to load data involves full refresh or incremental refresh and factors like transaction constancy, error management and performance maximization are taken into consideration. Transformation logic has been moved into the target system with the creation of ELT architectures, by using the compute capabilities of modern cloud data warehouses. This phase is typically scheduled to run during non-peak hours to avoid disrupting the performance of the warehouse [\(Nishanth R. M., 2019\)](#).

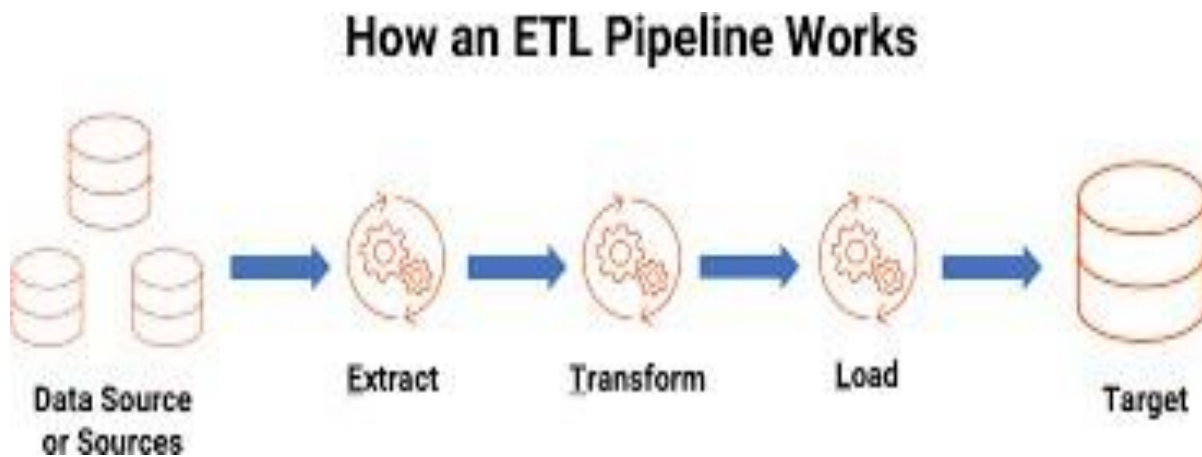


Figure 2.1: A simple representation of an ETL pipeline [\(Informatica, 2025\)](#)

2.3 Review of Batch Job Failures

Enterprise computing is still based on batch job processing, which processes scheduled data processing tasks, generates reports, synchronizes data, performs backup operations, trains machine learning models. It is essential to know how, why, and when batch job failures happen to sustain effective data pipeline processes and business continuity. Analyzed results show that job failure rates are significant in most High-Performance Computers (HPCs), and on average, a failed job often consumes more computational resources than a successful job [\(Yuan et al., 2012\)](#)

The resource usage patterns of failed and completed jobs can differ before termination,

especially where retry activity, task resubmission, scheduling delay, and abnormal consumption appear in the execution history. Large-scale studies of job failures show that failed jobs may consume considerable computation before failure, which makes early detection valuable for cost control and operational reliability ([Yuan et al., 2012](#)).

The frequency of failures may also vary across time windows because of resource contention, maintenance activities, dependency failures, and workload changes. Workload analysis in large heterogeneous GPU clusters shows that production traces can contain diverse job types and resource requirements, which supports the use of source-specific models rather than assuming that one feature schema can represent every execution environment ([Weng et al., 2022](#)).

2.3.1 Impacts and costs of Failed Batch Jobs

Batch job processing remains useful because large volumes of data can be processed at scheduled intervals, often during periods of lower system demand. The disadvantage is that a failed batch job can delay an entire reporting or analytics workflow, especially when the process runs outside normal working hours. This makes early warning important because the objective is not simply to record that a job failed, but to give engineers a chance to inspect abnormal conditions while the process is still running ([Chanda, 2024](#)).

2.3.2 Categories of Extract Load Transform Batch Job Failures

There are several root causes of batch job failures, and they can be classified into broad categories of resource-related, data-related, system-related and configuration-related. The failures associated with resources are because of the exhaustion of memory, CPU, throttling, disk space limitations, bandwidth limits in the network, and even failures in timeouts. Such failures usually happen due to the mistaking of resources obligations or rivalry with other tasks.

Data failures are associated with missing input files, corrupt records, mismatched schema, invalid values, duplicated records, and volumes that exceed expected processing capacity. These failures are challenging because an upstream defect can propagate into transformation, loading, reporting, and machine-learning stages. Data pipeline quality studies therefore emphasize that failure analysis should consider where in the pipeline the

issue occurs and how it affects later processing areas ([Foidl et al., 2024](#)).

System failures can include hardware faults, network interruptions, software bugs, platform instability, dependency timeouts, and infrastructure maintenance disruptions. In cloud and cluster environments, the interaction between applications, nodes, schedulers, and resource constraints makes failure prediction difficult because a job may fail for reasons that are only partially visible in application logs ([Jassas & Mahmoud, 2022](#)).

Configuration related failures are caused by parameters that are not properly set, unsuitable versions of software, lack of dependencies and inappropriate level of privileges and errors in configuration of schedulers. These failures are often attributed to flaws by people during job definition or environment configuration and hence justification and automated configuration control is necessary.

2.4 Review of Machine Learning Algorithms for Batch Job Failure Prediction
Machine learning represents a continuum of methods involving allowing systems to learn undirected data without being explicitly programmed. The paradigm is customarily divided into three major paradigms of learning, which apply to various contexts of problems and data access. Machine learning is a part of computer science that emanated from the study of pattern recognition and computational learning theory all in artificial intelligence ([Udousoro, 2020](#)). The most popular divisions of machine learning are discussed subsequently.

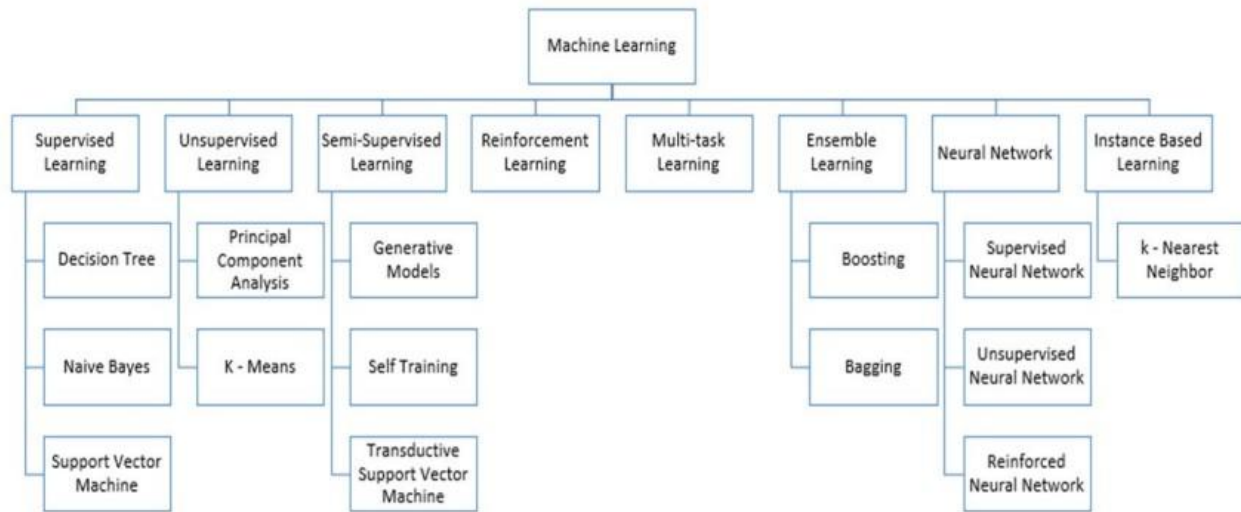


Figure 2.2: Machine learning Techniques

[\(Udousoro, 2020\)](#)

Supervised learning entails the training of models with labeled datasets whereby the input features are accompanied with known output labels. The model will learn to localize the input to the output by reducing errors in prediction on the training data. To anticipate future events, supervised machine learning algorithms use labelled examples to apply what they have learned in the past to fresh data [\(Shaveta, 2023\)](#). The paradigm performs well in cases where the historical labeled data is rich and the dependence between features and results can be acquired with examples. The trainer is required to learn from the training set with examples of labelled set which will be used to identify the unlabeled examples in the test set with highest possible accuracy [\(Udousoro, 2020\)](#). Supervised learning can be grouped further in two categories of algorithms, Regression and Classification algorithms [\(Shaveta, 2023\)](#). Classification algorithms are used to predict discrete values, whereas regression algorithms are used to predict continuous values [\(Breiman, 2001\)](#).

Unsupervised learning also operates on data sets of unspecified labels by exploring

patterns, structures or deviations present in the data. This learning technique does not produce classification but makes decisions that maximize rewards ([Udousoro, 2020](#)). The machine is trained using a set of unlabeled, unclassified, or uncategorized data, and the algorithm is required to respond independently to that data ([Shaveta, 2023](#)). Unsupervised learning does not need the presence of a person to oversee the model which may be beneficial in some situations. However, supervised learning models provide more accurate results because a programmer explicitly teaches the system what to search for in the data presented. Unsupervised learning, on the other hand, may be highly unexpected ([Naeem et al., 2023](#)). In the case of log analysis, unsupervised methods are the best when it comes to identifying new failure modes that were not in the training set and unequal system behavior without necessitating massive labeling.

Semi-supervised methods use both labeled and unlabeled data and are useful where labeled failure examples are limited but large quantities of normal logs are available. The main objective of semi-supervised learning is to reduce dependence on fully labeled datasets while still using the structure of available unlabeled data ([Reddy et al., 2018](#)). Semi-supervised log-based anomaly detection can be relevant to pipeline monitoring, but its results must be interpreted carefully because the absence of labeled failures can make validation difficult ([Yang et al., 2021](#)).

Reinforcement learning trains agents to make sequential decisions by interacting with an environment and receiving rewards or penalties. It is less central to this study than supervised learning because the present project is based on labeled success and failure outcomes rather than online reward optimization. However, reinforcement learning remains relevant to future work because it could support dynamic scheduling, resource allocation, and intervention policies after a failure-prediction system has been validated ([Naeem et al., 2020](#)).

2.4.1 Review of Log Analysis using Machine Learning

Log analysis is a critical task in modern computing environments, where systems generate vast amounts of logs from applications, servers, networks, security tools, and cloud platforms. Traditional manual or rule-based log analysis methods are no longer sufficient due to the volume, variety, and velocity of log data. Machine learning has

emerged as a powerful approach to automate and enhance log analysis, enabling more accurate detection of anomalies, prediction of system failures, and improved operational visibility.

Machine learning techniques help in parsing unstructured log data, identifying patterns, and detecting deviations that may indicate faults, intrusions, or performance issues. Unsupervised learning approaches such as clustering and anomaly detection models are especially useful because logs often lack labeled data. These models learn the normal behavior of log sequences and flag unusual patterns. Supervised learning methods are also applied in scenarios where labeled data is available, enabling the classification of log events or the prediction of known error types. Deep learning techniques, particularly Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTMs), and Transformers, have further improved log analysis by modeling sequential dependencies within logs.

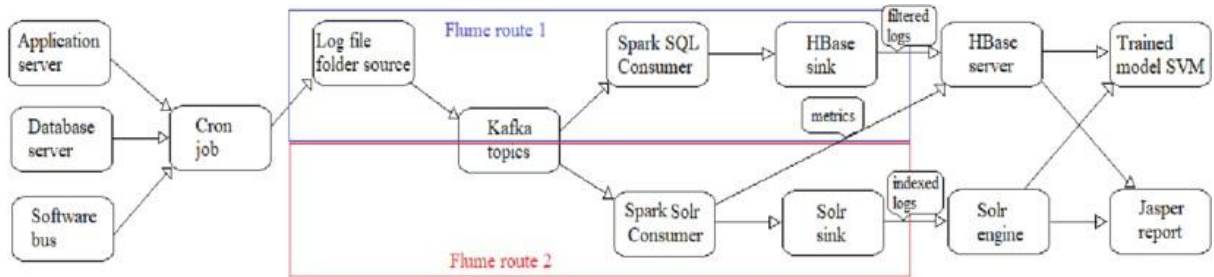


Figure 2.3: Workflow for Log Analysis and Prediction

2.5 Review of Existing Models and Gaps

The knowledge of the current methods of pipeline failure prediction and log-based anomaly detection allows us to give the understanding of the new solution novelty and contributions to the current work. The field of research consists of a wide variety of methodologies consisting of both statistical methods, as well as traditional machine learning and deep learning implementations.

2.5.1 Analysis of Job Failure and Prediction Model for Cloud Computing using

Machine Learning

This study focused on predicting failed jobs before they occur to enhance resource consumption and cloud-based efficiency. It also aimed to study various cloud traces to find the correlation between failed and successful jobs. Batch job properties like job priority, scheduling class, resource request was extracted. Three publicly available traces: Google cluster, Mustang and Trinity were used to evaluate the proposed model. The study analyzed both successful and failed jobs using various machine learning algorithms: Decision Trees (DT's), Random Forest (RF), k -Nearest Neighbors (KNN), Quadratic Discriminant Analysis (QDA), Gradient Boosting (GB), XGBoost and Naïve Bayes (NB), while applying the proposed model to the three different cloud traces. The proposed model resulted in Decision Trees, Random Forest and XGBoost algorithms achieving the highest accuracy, precision, recall and F1-score. It also proved that jobs requesting too little memory were prone to crashing.

This research thoroughly analyzed job failure prediction in cloud computing using logs, but specific data pipelines or ETL workflows were not accounted for. It also relied heavily on the job metadata without analyzing actual log content or error patterns. This led to the models predicting failures at job submissions, not during the execution when early warnings could have prevented failures.

2.5.2 DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning

DeepLog was the first neural network that applied deep learning to log sequences to detect anomalies in logs with the help of LSTM networks ([Du et al., 2017](#)). The system considers logs as natural language patterns and uses LSTM to extract patterns in execution paths. This study proposed a deep neural network model utilizing LSTM, to model system logs as natural language sequences. It implicitly captured the potentially non-linear and high dimensional dependencies among log entries from the training data that correspond to normal system execution paths. The system achieved 88.9% accuracy where top predictions matched actual log keys and 96.1% of predictions were within the top two predictions. DeepLog was deployed in production at Huawei Cloud to analyze terabytes of daily log data using Kafka as a streaming channel or Flink for distributed log preprocessing.

Long Short-Term Memory models are very extensive in the sense that they need a lot of training data which may not be available for new or evolving pipelines. The system only accounted for general system logs and not specifically optimized batch jobs with their unique properties. This research mainly utilized log text for predicting failures, inconsequentially it ignored CPU, memory Input/Output (I/O) metrics which are also crucial for batch job failures.

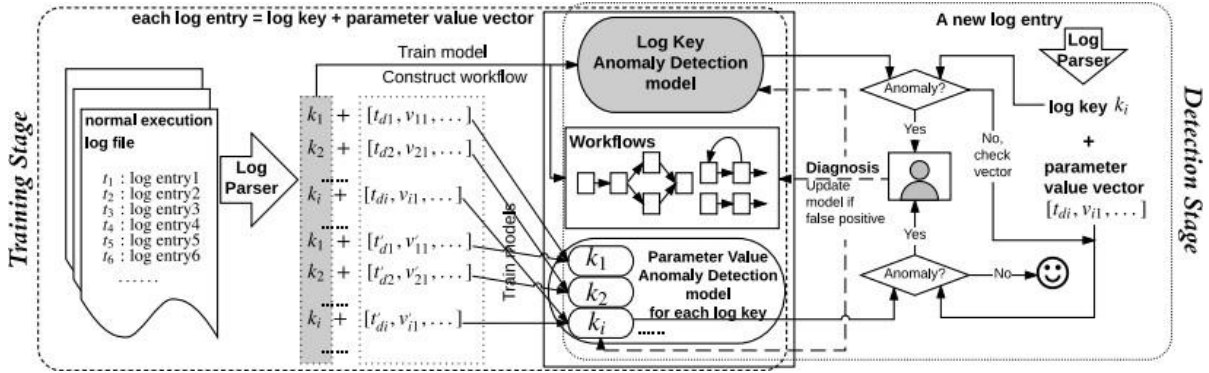


Figure 2.2: DeepLog Architecture (Zou & Petrosian, 2020)

2.5.3 Cloud failure prediction based on traditional machine learning and deep learning This paper made a comprehensive comparison and model evaluation of predictive models for job and task failures. Five machine learning algorithms and three deep learning algorithms were used to train the datasets. A benchmark dataset called Google cloud traces was used for the training and testing of models.

This research established that in the case of job failure prediction, Extreme Gradient Boosting produced the best model where the disk space request and CPU request are the most important features that influence the prediction. It also established that Decision Tree and Random Forest algorithms produce the best models where the priority of the task is the most important feature for both models. Logistic regression model was also discovered to be the most scalable compared to others.

2.6 Gaps Identified in Literature

Although much work has been done on the prediction of job failures in cloud computing

and prediction of log anomalies in general software systems, little to no solutions are created to tackle the specifics of the issues in data pipeline batch jobs. Data pipelines have complicated dependencies between jobs, differing data quality demands, diverse data origins, and particular failure strategies in regard to data processing as opposed to merely computational execution. A combined job-level prediction of failures, and log analysis based on data pipeline characteristics is a significant gap that can be addressed.

The current methods of predicting pipeline failures and analyzing the log have serious gaps, although the research on that matter is very large. The majority of research on log anomaly detection tests models on open benchmark datasets which might not be representative of real-world production data pipeline logs in terms of diversity and complexity. Practical pipelines may be based on heterogeneous systems, application specifics, and organization unique settings which vary significantly with standardized benchmark environments.

The capability of real-time prediction is usually absent or of lower priority. Numerous systems of research work on offline analysis or batch detection based on evaluation on historical data. Low-latency prediction is needed by production systems to detect potential failures before they happen, as this would permit the production system to take preventive measures, which puts intense computational efficiency requirements.

CHAPTER THREE

RESEARCH METHODOLOGY

3.1 Preamble

This chapter presents the methodology used to develop, demonstrate, and evaluate the log analysis and failure prediction system. The project is an applied software engineering and machine learning study: it is concerned with building a working monitoring plugin, training source-specific predictive models, and presenting the system as a visual pipeline monitoring workflow rather than as terminal-only output.

The methodology deliberately separates five layers of evidence. The first layer is the offline training evidence from stored datasets and model artifacts. The second layer is the ETL demonstration pipeline that emits process logs while it runs. The third layer is the predictor plugin, which ingests logs, extracts features, classifies risk, and generates reports. The fourth layer is the Streamlit dashboard, which shows the operator-facing workflow. The fifth layer is the Apache Airflow DAG, which shows how the workflow can be represented as orchestrated tasks.

This separation is important for academic defensibility. The dashboard can be visually convincing, but the claims in this report are tied to code, saved model artifacts, evaluation files, and explicitly described demonstration behavior. Where the implementation uses synthetic presentation calibration to make LOW, MEDIUM, and HIGH states visible during a defence session, the report identifies that behavior as demonstration behavior and does not treat it as a new measured probability.

The methodology is also framed around the practical problem that batch job failure prediction must be useful before a job reaches its terminal state. For that reason, the system was not designed only as an offline notebook experiment. It was designed to connect offline training artifacts with a live monitoring surface where log lines, feature summaries, risk decisions, and operator actions can be observed during execution. This makes the methodology suitable for a final-year software engineering project because it evaluates both model behavior and the usability of the monitoring workflow.

A further methodological concern was traceability. Each major claim in the implementation is

connected to an artifact that can be inspected, such as source code, saved model files, metrics files, comparison charts, generated reports, dashboard views, or the Airflow DAG. This reduces the risk of presenting the dashboard as a visual mock-up. The dashboard is instead treated as the interface through which stored evidence and runtime evidence are made understandable to an examiner.

The project therefore uses a layered evidence strategy. Public and local datasets support model development; the ETL demonstration supports live execution; the predictor plugin supports reusable ingestion and risk classification; the dashboard supports visualization; and the report generator supports post-run auditability. This layered strategy is consistent with the study aim because batch job failure prediction is not only a statistical problem but also an operational decision-support problem.

3.2 Research Design

The research design is an applied experimental design supported by prototype-based software development. The experimental part covers data loading, feature extraction, model training, model comparison, and held-out evaluation. The prototype part covers pipeline execution, live attachment, streaming analysis, intervention decisions, report generation, dashboard rendering, and Airflow orchestration.

The main predictive technique used in the implemented system is XGBoost. XGBoost was selected because boosted trees can model nonlinear relationships among heterogeneous operational indicators such as error counts, warning rates, retry activity, task failure counts, scheduling delays, and sampled resource metrics. The project still compares XGBoost with logistic regression, decision tree, random forest, histogram gradient boosting, and a majority baseline because the defence claim is stronger when the selected model is considered against alternatives.

The development process followed iterative refinement. Earlier iterations focused on terminal monitoring and saved log files. Later iterations added the Streamlit dashboard, live ETL integration, predictor plugin, HIGH-risk review controls, integer presentation percentages for the dashboard, source-specific model artifacts for Alibaba and Google traces, and a deploy-ready Streamlit configuration for Render. Each iteration preserved the distinction between evidence produced by the trained model and behavior designed for demonstration.

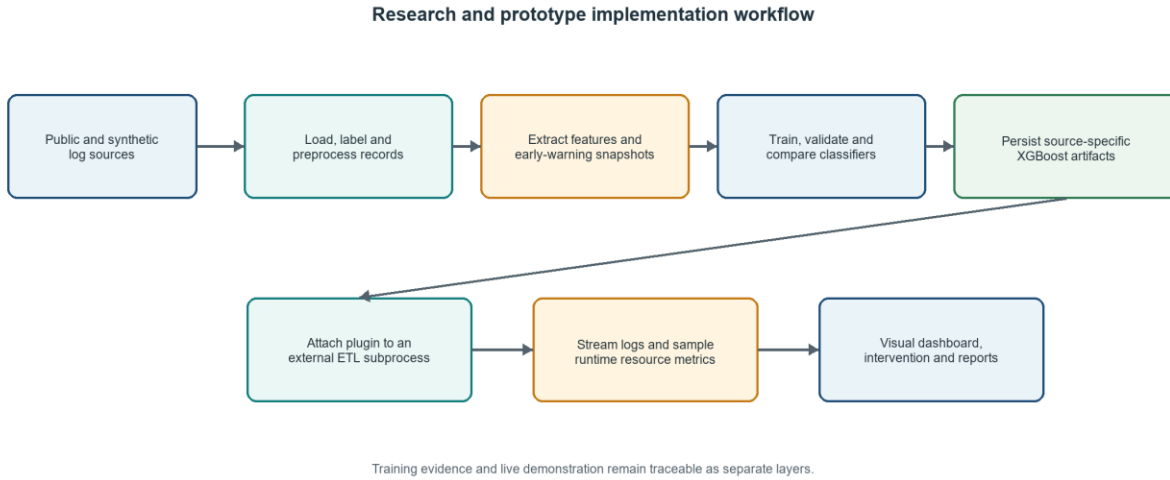


Figure 3.1: Research and prototype implementation workflow

Table 3.1: Methodological alignment with project objectives

Objective	Method used in the implementation	Evidence retained in the project
Identify failure patterns in batch jobs	Log severity counts, keyword signals, retry indicators, failed task counts, resource and scheduling features were extracted from logs and workload traces.	feature_extractor.py; data_loader.py; saved metrics and feature importance charts
Compare machine learning algorithms	A comparison routine evaluated logistic regression, decision tree, random forest, histogram gradient boosting, XGBoost, and a majority baseline.	model_comparison.py; outputs/model_comparison.csv; comparison_alibaba and comparison_google2019 folders
Develop a prediction model for monitoring	Source-specific XGBoost artifacts were trained and saved. The live dashboard uses the ETL/HDFS-compatible artifact for demonstration.	models/xgboost_pipeline_monitor_etl.pkl; models/xgboost_pipeline_monitor_alibaba. pkl; models/xgboost_pipeline_monitor_google2019.pkl
Evaluate with precision, recall and F1	Evaluation files store accuracy, precision, recall, F1, ROC-AUC, PR-AUC, specificity, MCC, log loss, and confusion-matrix counts.	outputs/metrics.json; Chapter Four result tables
Represent the system visually	Streamlit renders stage boards, logs, risk cards, review controls, model comparison charts, reports, and system maps.	dashboard_app.py

The applied experimental part of the design followed a supervised-learning workflow because the available data can be organized into successful and failed outcomes. Supervised learning is suitable when historical labels exist and the objective is to learn a mapping between execution features and a failure outcome. This justified the use of precision, recall, F1-score, ROC-AUC, PR-AUC, confusion-matrix counts, and related metrics, since the research question concerns the

detection of failure risk rather than only the description of log events.

The prototype-development part of the design followed an incremental build-and-test approach. At each stage, a working software capability was added and then checked against the project objectives. The first capability was offline feature extraction and model training. The next was model comparison. After that, the work progressed to the predictor plugin, live ETL runner, dashboard, report generator, and Airflow DAG. This sequence helped ensure that the final system was not built around a single screen but around a complete monitoring flow.

XGBoost was selected as the main deployed model because gradient boosted decision trees are effective for tabular operational features and can model nonlinear interactions among counts, rates, timing indicators, resource variables, and failure keywords ([Chen & Guestrin, 2016](#)). Review literature also supports the continued relevance of XGBoost in applied machine-learning classification tasks ([Arif et al., 2023](#)). The selection was methodologically cautious because XGBoost was not treated as automatically superior. It was compared with logistic regression, decision tree, random forest, histogram gradient boosting, and a majority baseline, so that the final interpretation could distinguish the selected implementation from the broader comparison evidence.

The comparison design also reflects the diversity of machine-learning methods. Logistic regression provides a simple linear baseline ([Maalouf, 2011](#)). Decision-tree methods provide interpretable rule-based splits ([Quinlan, 1986](#)), and applied predictive work continues to show their usefulness in classification tasks ([Matzavela & Alepis, 2021](#)). Recent survey evidence also describes decision trees as a continuing foundation for interpretable machine-learning applications ([Mienye & Jere, 2024](#)). Random forests provide ensemble averaging over trees ([Breiman, 2001](#)), while support-vector approaches represent margin-based classification traditions ([Cortes & Vapnik, 1995](#)). K-nearest neighbor was considered in the wider literature review as a supervised-learning method, but it was not selected for the implemented comparison because the project focused on models better suited to the saved tabular and tree-based workflow ([Suyal & Goyal, 2022](#)).

3.3 Population of the Study and Data Sources

The population of the study is made up of execution records that describe successful and failed computational jobs or pipeline processes. Because pipeline failures can occur in different

environments, the project uses more than one evidence source. The HDFS-compatible source supports log-based feature extraction and the live ETL demonstration. The Alibaba PAI GPU trace and Google Borg 2019 trace provide production-style workload evidence. The synthetic ETL pipeline provides controlled runtime behavior for the dashboard and plugin demonstration.

The HDFS-compatible model is used for the live ETL dashboard because the live runner adapts external pipeline messages into an HDFS-style stream with a stable job identifier. The Alibaba and Google artifacts are not forced into this live dashboard model; they remain separate benchmark artifacts because their schemas are workload-trace schemas rather than HDFS log-line schemas.

The raw compressed Alibaba and Google trace archives were moved out of the deployable repository before Render deployment preparation. This does not affect the deployed demonstration because the deployed dashboard uses saved model artifacts, small sample data, and saved comparison outputs. Retraining on the large compressed traces remains a local research task, not a cloud deployment requirement.

Table 3.2: Dataset sources and project role

Source	Local role		Why it was used	Deployment status
HDFS-compatible logs	Primary log schema and live ETL-compatible model		Supports log-line grouping, severity counts, failure keywords, and early warning snapshots.	Raw HDFS.log excluded from deployment; trained model retained.
Alibaba PAI GPU trace	Bounded production-trace benchmark		Provides workload-level evidence from task and job records.	Raw archive excluded from deployment; model and comparison summary retained.
Google ClusterData2019	Borg Bounded production-trace benchmark		Provides large-scale cluster workload evidence with a different schema.	Raw archive excluded from deployment; model and comparison summary retained.
Synthetic ETL pipeline	Live defence demonstration		Provides a controllable pipeline with extract, validate, transform, load, and report stages.	Included through source code and sample_orders.csv.

The population was not treated as human respondents, survey participants, or manually selected opinions. It was treated as operational evidence generated by computational jobs and data

pipeline processes. This distinction is important because the unit of analysis is a job, task, log sequence, or observation window rather than a person. The study therefore uses records that describe execution behavior, severity levels, resource usage, task outcomes, and observable progress through pipeline stages.

The public workload traces were used to broaden the evidence base beyond the small ETL demonstration. The Alibaba trace provides heterogeneous GPU-cluster workload evidence and is appropriate for studying large-scale job behavior ([Alibaba Group, 2022](#)). Workload analysis of large heterogeneous GPU clusters further supports the importance of examining source-specific execution behavior ([Weng et al., 2022](#)). The Google Borg 2019 trace provides another production-style workload source with a different schema and operational context ([Google, n.d.](#)). These sources support the methodological decision to retain source-specific models rather than force all observations into one artificial schema.

The HDFS-compatible source and the ETL demonstration have a different role. They support log-line grouping, streaming feature extraction, and visible risk movement inside the dashboard. Public log collections such as Loghub show why system logs are useful for AI-driven log analytics, but the live demonstration still needed a controllable pipeline that could run safely during defence ([Zhu et al., 2023](#)). The ETL demonstration therefore acts as the bridge between research evidence and examiner-facing software behavior.

3.4 Sampling Technique

A bounded purposive sampling technique was used because the public trace files are large and the project had to remain reproducible on a local development computer. The bounded samples were large enough to train and evaluate the source-specific models while avoiding the deployment problem of committing multi-gigabyte raw logs to GitHub.

The Alibaba loader scanned job-level and task-level records and constructed a bounded labeled sample of 5,000 jobs. The sample contained 273 failures and 4,727 successful jobs. The Google 2019 loader constructed a bounded sample of 10,000 labeled records, including 2,299 failures and 7,701 successes. The ETL demonstration dataset was generated through scenario loops, producing 100 ETL records and 400 early-warning snapshots in the stored build used for reporting.

The project does not claim that the bounded samples represent the entire Alibaba or Google traces. The correct interpretation is that they provide additional benchmark evidence for the prototype. Larger experiments are recommended in Chapter Five, but the current report only states the evidence available in the saved artifacts.

Purposive bounded sampling was used because the purpose of the project was to build and demonstrate a defensible prototype, not to exhaust every row in very large public traces. This decision made the experiments feasible on a local development machine and prevented the deployed repository from depending on multi-gigabyte raw archives. The bounded samples were selected through code-based loaders, which means the sampling logic can be reviewed and repeated instead of being hidden in manual spreadsheet editing.

The sampling approach also considered class imbalance. Failure records are often less frequent than successful records, and this can make a model appear strong if accuracy is the only metric used. For this reason, the study preserved the failure counts for each bounded sample and used metrics that are more sensitive to the minority class. Precision, recall, F1-score, PR-AUC, and MCC were therefore included alongside accuracy.

The ETL demonstration sample had a different sampling purpose. It was not intended to represent all production pipelines. It was designed to create repeatable scenarios that show LOW, MEDIUM, and HIGH risk states, stage transitions, warning messages, recovery attempts, and report generation. This makes the demonstration sample valuable for showing system behavior, while the public traces provide broader benchmark evidence.

3.5 Instrumentation and Development Tools

The principal research instrument is the implemented software prototype. The prototype consists of Python modules for loading data, extracting features, training models, comparing classifiers, monitoring live logs, adapting external process output, generating reports, rendering the dashboard, and defining the Airflow DAG.

Python was used because it provides mature libraries for data manipulation, machine learning, dashboard development, and process automation. The project dependencies include XGBoost, scikit-learn, pandas, NumPy, Matplotlib, Seaborn, Streamlit, and psutil. The models are stored under the models directory, while reproducible evaluation outputs and comparison charts are

stored under outputs.

Table 3.3: Implementation modules used as research instruments

Module	Main responsibility	Defence relevance
data_loader.py	Loads HDFS, Google, Alibaba, and Google 2019 records.	Shows how different data sources enter the system.
feature_extractor.py	Converts raw records and streaming logs into numeric features.	Explains model inputs and early-warning evidence.
model.py	Trains XGBoost and stores evaluation artifacts.	Supports the model-development objective.
model_comparison.py	Compares baseline and candidate classifiers.	Supports the algorithm comparison objective.
pipeline_monitor.py	Wraps streaming prediction and risk calibration.	Connects model output to LOW, MEDIUM, and HIGH risk.
predictor_plugin.py	Provides the plugin-facing API and report payload.	Represents the project as an attachable monitoring plugin.
live_pipeline_runner.py	Runs external commands and streams logs into the plugin.	Demonstrates live attachment to a pipeline process.
etl_pipeline_demo.py	Implements the extract, validate, transform, load, and report demo pipeline.	Makes the defence demo feel like a recognizable pipeline.
dashboard_app.py	Renders the Streamlit dashboard.	Provides the visual representation required for defence.
dags/pipeline_failure_monitor_demo.py	Defines the Airflow DAG.	Provides orchestration evidence.

The implemented software prototype served as the main research instrument because it produced the model artifacts, monitoring outputs, dashboard views, and reports used in the analysis. Python was appropriate because the ecosystem includes mature data and machine-learning libraries, including pandas, NumPy, scikit-learn, and XGBoost. Scikit-learn was especially useful for splitting data, training comparison models, and computing metrics ([Pedregosa et al., 2011](#)).

The evaluation tools were selected to match the project objectives. F1-score was included because it balances precision and recall and is more informative than accuracy when failures are less common ([scikit-learn developers, n.d.-a](#)). Stratified and grouped validation concepts were also important because related snapshots from the same job should not be carelessly treated as fully independent records across training and testing partitions ([scikit-learn developers, n.d.-b](#)).

Streamlit was used as the dashboard framework because it allows Python-based data applications to present controls, charts, logs, tables, and downloadable outputs in a browser interface ([Streamlit, n.d.](#)). Apache Airflow was included as orchestration evidence because its DAG model

represents a pipeline as ordered tasks with configurable execution behavior ([Apache Software Foundation, n.d.](#)). These tools helped convert the machine-learning experiment into a practical monitoring workflow.

3.6 Data Collection and Preprocessing

The project uses secondary public traces and locally generated ETL evidence. Public traces are read through code-based loaders rather than by manually editing the raw files. This is important because the preprocessing steps can be repeated, reviewed, and adjusted without changing the raw dataset.

The HDFS loader parses log lines into date, time, process identifier, severity level, component, and content fields. The Alibaba loader reads compressed archive members and combines job-level and task-level evidence. The Google 2019 loader reads the locally available prejoined trace export and maps workload fields into numeric features. The ETL demonstration produces structured logs directly from the running pipeline stages.

Preprocessing also includes label handling. Success and failure labels are used for supervised training and evaluation. Terminal outcome fields are not treated as live predictive inputs. This distinction protects the project from a leakage problem where the model would appear accurate only because it was given the answer after the job had already finished.

Data collection was performed through reproducible loaders and generated runtime evidence. This distinction matters because the public traces and the live ETL demonstration entered the system through different paths. The public traces were read as stored datasets, while the ETL evidence was produced dynamically by a running process. Both forms of evidence were useful, but they required different preprocessing decisions.

For stored datasets, preprocessing focused on schema interpretation, missing-value handling, label construction, type conversion, and feature normalization where needed. For live logs, preprocessing focused on line parsing, timestamp capture, severity extraction, job identification, component detection, and message normalization. The system therefore separates workload-trace preprocessing from streaming log preprocessing instead of pretending that every source is the same type of data.

The avoidance of label leakage was an important preprocessing control. Terminal fields that

directly reveal whether a job succeeded or failed were used for labels during supervised training, but they were not treated as live predictive inputs. This protects the experiment from producing artificially high scores that would not be available during real monitoring. In a prediction system, features should represent warning evidence available before or during execution, not the final answer after the job ends.

Preprocessing also made the project more reproducible. The loaders, feature extractors, and demonstration scripts document how each observation becomes a model-ready record. This makes the research stronger than a manual workflow because another reviewer can inspect the code path, repeat the preparation, and understand why particular fields were included or excluded.

3.6.1 HDFS-Compatible Log Preparation

The HDFS-compatible preparation groups lines by a stable block or job identifier. For live ETL demonstration, external messages are adapted into this structure so that streaming feature extraction can work with a predictable schema. The adapter preserves timestamp, level, component, message, stage, and job identity information where available.

The HDFS-compatible preparation was built around grouped execution evidence. Each log line was associated with a stable identifier so that the feature extractor could summarize a job or observation window over time. This grouping is important because isolated log lines may be harmless, while a sequence of repeated warnings, retries, and errors can indicate a growing failure condition.

The live ETL adapter used the same principle. It converted external pipeline messages into a predictable structure containing timestamp, severity, component, stage, message, and job identity information. This allowed the monitoring logic to update features continuously without requiring the ETL pipeline itself to be rewritten as a machine-learning script.

3.6.2 Alibaba PAI Preparation

The Alibaba preparation focuses on job and task summaries. It derives counts and rates such as task count, failed task count, scheduling delay, resource requests, and terminal job state. These values support workload-level failure prediction rather than free-text log anomaly detection.

The Alibaba preparation focused on workload-level records rather than natural-language log

messages. Job and task information was summarized into fields such as task counts, unsuccessful task counts, scheduling delay, resource requests, and terminal status. These fields are suitable for tabular prediction because they describe the structure and outcome of computational work in a cluster environment.

The preparation also respected the size of the raw trace. Large compressed archives were not committed to the deployable repository. Instead, the project retained model artifacts and comparison summaries so that the dashboard could remain lightweight while the training evidence remained explainable in the report.

3.6.3 Google 2019 Preparation

The Google 2019 preparation uses a prejoined local export derived from the published Borg trace schema. The implementation treats it as a separate source because its feature schema differs from HDFS and Alibaba. This prevents a false claim that one universal feature schema was trained across all sources.

The Google 2019 preparation treated the Borg trace as a separate benchmark source because its schema differs from the HDFS-compatible and Alibaba sources. The preprocessing mapped available workload fields into numeric features and retained labels suitable for failure prediction. This approach avoided the methodological error of claiming that one universal model had been trained across incompatible schemas.

The Google sample also served as a stress test for interpretation. Its results were more modest than the controlled ETL experiment, which helped the report present a balanced conclusion. The model should therefore be understood as a prototype and benchmarked implementation, not as a universally validated production predictor.

3.6.4 ETL Demonstration Preparation

The ETL demonstration uses `data/sample_orders.csv` as a small deployable input file. The running ETL process emits stage-level logs as it extracts records, validates data, transforms accepted rows, loads output, and writes a report summary. Because this file is small and reproducible, it is suitable for Render deployment and defence demonstration.

The ETL demonstration was prepared to be small, repeatable, and safe to run during a defence session. It reads a compact sample order file, executes recognizable ETL stages, and emits log

messages that represent normal progress, warnings, recoveries, and risky conditions. This gives the examiner a concrete pipeline to observe instead of only a static dataset.

Because the ETL demonstration is controlled, it is also transparent. The scenarios are intentionally selectable so that the dashboard can show different operational states. The report therefore separates demonstration behavior from model-evaluation evidence, which is important for academic honesty and for avoiding overstated production claims.

3.7 Feature Engineering

Feature engineering is the bridge between raw operational evidence and machine learning. The implementation does not pass raw logs directly into XGBoost. Instead, it summarizes each job or observation window into structured numeric features that reflect severity, frequency, timing, workload structure, resource context, and repeated failure signals.

For HDFS-compatible logs, feature extraction includes total line counts, severity counts, rates of warnings and errors, fatal indicators, retry indicators, exception keywords, failed/stopped/timeout keywords, unique components, unique process identifiers, maximum error burst, and timing information. These features support early-warning prediction because they can be updated as new lines arrive.

For Alibaba and Google workload traces, feature extraction emphasizes workload structure. Examples include task counts, unsuccessful task counts, scheduling delay, requested CPU, requested memory, requested GPU, priority fields, sampled runtime usage, and status-derived labels. These source-specific features are kept separate from HDFS-compatible features.

Table 3.4: Feature engineering categories

Feature category	Examples	Reason for inclusion
Severity profile	INFO, WARN, ERROR, FATAL, DEBUG counts and rates	Shows whether the execution stream is becoming operationally unhealthy.
Failure keywords	exception, failed, timeout, retry, stopped, invalid	Captures text patterns commonly associated with failure or recovery attempts.
Burst behavior	maximum consecutive error count	Distinguishes isolated warnings from clustered degradation.
Execution diversity	unique components and process identifiers	Shows whether many components are involved in the incident.
Workload structure	task count, failed task count, scheduling delay	Represents job-level complexity and unsuccessful work.
Resource context	CPU, memory, disk, GPU request, usage snapshot	Captures operational pressure that can contribute to failure risk.
Temporal evidence	elapsed time and early-warning snapshot	Supports prediction before terminal failure.

Feature category	Examples	Reason for inclusion
	fraction	

Feature engineering was treated as a central research activity because the quality of the model depends on the quality of the operational evidence supplied to it. Raw log text can be difficult for a tabular model to interpret directly, so the system converts logs and workload records into numeric signals. These signals represent severity, repetition, timing, resource context, and workload structure.

For log-based evidence, the strongest features are those that can be observed while a process is still running. Counts of warnings and errors, ratios of abnormal messages, repeated exception keywords, timeout indicators, retry messages, and error bursts are useful because they can change before a final failure occurs. This supports early warning rather than post-mortem classification.

For workload-trace evidence, the important features are different. Task counts, unsuccessful task counts, scheduling delay, requested resources, usage snapshots, and priority fields describe how a job interacts with a cluster. These features are consistent with studies showing that job outcomes can be influenced by resource use, scheduling behavior, and task-level instability ([Yuan et al., 2012](#)). Cloud-job failure prediction research also supports the value of relating job behavior to machine-learning features ([Jassas & Mahmoud, 2022](#)).

The implementation deliberately kept feature sets source-specific. A feature such as failed task count is meaningful in a workload trace, while a feature such as maximum consecutive ERROR lines is meaningful in a log stream. Keeping these feature sets separate improved methodological validity because it allowed each model to learn from the evidence type that actually exists in its data source.

3.8 Model Development and Validation

The model-development process follows a repeatable sequence: load labeled records, extract features, split data, train candidate models, evaluate held-out predictions, generate charts, and persist the selected model artifact. The selected deployed model family is XGBoost, but comparison artifacts are retained so that the report can show how XGBoost performed against alternative classifiers.

Separate model artifacts are retained for the HDFS-compatible/ETL demonstration, Alibaba, Google 2019, and legacy/default HDFS-compatible model. This is a key methodological decision. A single merged model would require a consistent cross-source feature schema and cross-domain validation. The current implementation does not make that claim.

Validation uses held-out performance metrics and comparison outputs. Where repeated snapshots are generated from one execution, the grouping logic is intended to reduce leakage by keeping related snapshots from being treated as independent evidence across training and testing. The reported metrics include confusion-matrix counts, precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, specificity, log loss, and inference latency.

Table 3.5: Stored model artifacts

Artifact	Role	Use in defence
xgboost_pipeline_monitor.pkl	Legacy/default HDFS-compatible model	Kept as an earlier artifact and fallback path.
xgboost_pipeline_monitor_etl.pkl	ETL/HDFS-compatible live demonstration model	Used by the dashboard for the live ETL demo.
xgboost_pipeline_monitor_alibaba.pkl	Alibaba benchmark model	Shows source-specific benchmark training.
xgboost_pipeline_monitor_google2019.pkl	Google 2019 benchmark model	Shows second production-trace benchmark training.

The model-development process began with a baseline expectation: the selected model should perform better than a majority-class classifier and should also be interpreted against alternative machine-learning methods. This was necessary because failure prediction can be distorted by class imbalance. A majority baseline can produce high accuracy by always predicting success, but such a model would be operationally useless because it would miss the failure class.

Validation therefore emphasized failure-class performance. Recall shows how many actual failures are detected; precision shows how many predicted failures are truly failures; and F1-score provides a combined view when both missed failures and false alarms matter. PR-AUC was included because it is useful in imbalanced classification settings where the positive class is operationally important.

ROC-AUC, specificity, MCC, log loss, and inference latency were also retained because each metric answers a different question. ROC-AUC describes ranking ability, specificity describes how well successful jobs are not falsely flagged, MCC gives a balanced single score from the confusion matrix, log loss reflects probability quality, and latency shows whether prediction is

fast enough for monitoring use.

The saved model artifacts are part of the validation evidence because they make the experiment repeatable. The project stores the trained model files, metrics, comparison outputs, and charts rather than relying only on screenshots. This helps distinguish the implemented model pipeline from a presentation-only dashboard.

The validation process was also designed to examine whether the model is useful in an operational setting, not only whether it can produce a high aggregate score. In batch-job monitoring, a false negative can allow a failing job to continue unnoticed, while a false positive can interrupt work unnecessarily. The evaluation therefore considered the confusion matrix as a decision-support object. True positives represent correctly detected risky jobs, false negatives represent missed risk, false positives represent unnecessary warnings, and true negatives represent stable executions that were correctly left alone.

The use of a majority baseline was important because failure prediction datasets are often imbalanced. If most jobs succeed, a naive classifier can appear accurate by predicting success for nearly every case. Such a classifier would not satisfy the aim of this project because it would provide little support during the rare but important moments when failure is likely. Comparing against the majority baseline therefore helped show whether the trained models learned meaningful warning patterns rather than only the dominant class distribution.

The validation outputs were saved as project artifacts so that the results could be revisited during writing, defence, and future improvement. This includes metrics, comparison tables, charts, and model files. Retaining the artifacts makes the study more transparent because the reported values are not isolated numbers typed into the report after the fact. They are connected to reproducible files created by the implementation.

Model validation was also treated as source-specific. The ETL/HDFS-compatible model, Alibaba model, and Google 2019 model were not collapsed into one overall score because each source has different fields and different operational meaning. This decision improves the honesty of the evaluation. A strong result in the controlled ETL setting supports the live demonstration, while weaker benchmark results identify where additional data and tuning are still needed.

3.9 System Architecture and Implementation

The system architecture is organized around an attachable monitoring idea. A pipeline process can run independently while the monitoring layer observes logs, samples runtime metrics, updates features, applies a prediction model, classifies risk, and records the monitoring session. The project therefore demonstrates both the software workflow and the decision-support workflow.

The live attachment architecture is different from a static notebook. It uses a subprocess runner, a log adapter, a streaming extractor, a predictor plugin, and a dashboard state layer. This makes the implementation closer to a realistic monitoring plugin because the pipeline and the predictor are not the same object.

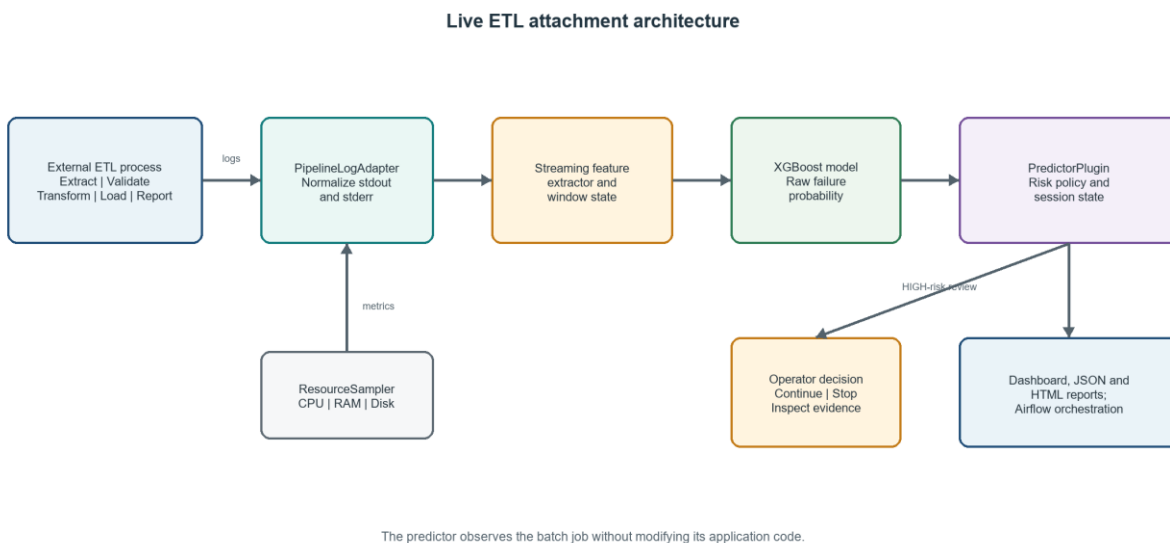


Figure 3.2: Live ETL attachment architecture

The architecture follows a separation-of-concerns pattern. The ETL process generates operational events, the runner captures those events, the adapter normalizes them, the feature extractor summarizes them, the model estimates risk, the plugin stores alerts and reports, and the dashboard presents the session. This separation makes the system easier to reason about because each component has a defined role.

The attachable monitoring idea is important because many real pipelines already exist before a prediction system is introduced. Requiring every pipeline to be rewritten would reduce adoption. The live runner therefore demonstrates how a predictor can sit beside an external command and observe stdout, stderr, and runtime information without turning the original process into a

notebook.

The architecture also supports auditability. A prediction without supporting logs, stage information, or a report is difficult for an operator to trust. The implemented system therefore stores risk counts, high-risk items, recent logs, resource summaries, stage summaries, and report outputs so that the decision can be reviewed after the session.

3.9.1 External ETL Batch Process

The ETL process contains extract, validate, transform, load, and report stages. It reads sample order data, validates record fields, transforms accepted records, writes a curated output, and emits progress logs. Scenario controls such as normal, warning, recovering, risky, and mixed allow the demonstration to exercise different risk paths.

The external ETL batch process was designed around recognizable data-engineering stages: extraction, validation, transformation, loading, and reporting. Each stage emits progress messages so that the monitoring layer can observe normal movement and abnormal behavior. This makes the demonstration more realistic than a random event generator because it follows a process structure that data engineers can recognize.

The scenario controls provide repeatability. A normal scenario can show stable execution, while warning, recovering, risky, and mixed scenarios can show different alert paths. These controls help the examiner understand the intervention logic without requiring an unpredictable production incident to occur during the defence.

3.9.2 Live Attachment Runner

The live runner starts an external command, captures stdout and stderr, normalizes each line, and submits the adapted record to the predictor plugin. It also uses psutil when available to collect CPU, memory, and disk information. This allows the dashboard to show runtime context instead of only text logs.

The live attachment runner is responsible for connecting the external process to the monitoring layer. It starts the command, captures output streams, preserves recent lines, and forwards normalized records to the predictor plugin. This design shows that the prediction system can receive evidence from a process as it runs, which is closer to real monitoring than training and testing only from a saved CSV file.

Runtime context is also included where available. CPU, memory, and disk samples help the dashboard show whether abnormal messages are accompanied by resource pressure. These samples are not treated as a complete infrastructure observability platform, but they improve the practical value of the prototype by adding context around the prediction.

3.9.3 Predictor Plugin

The predictor plugin is the reusable wrapper around the monitor. It ingests lines, stores alerts, detects HIGH risk, prepares review state, generates report payloads, and writes JSON and HTML reports. It gives the project a plugin-style interface rather than leaving prediction buried inside one script.

The predictor plugin is the point where the monitoring workflow becomes reusable. Instead of scattering prediction logic across the dashboard, ETL script, and report generator, the plugin receives normalized records, updates the monitor, stores alerts, tracks high-risk states, and prepares report payloads. This gives the project a clearer software boundary.

The plugin design also supports policy-based behavior. The same prediction can be used in advisory mode, review mode, or auto-stop mode depending on the operational setting. This is important because not every HIGH-risk prediction should automatically terminate a process; some environments require human approval, while others may prefer automated stopping when the cost of continuing is too high.

3.9.4 Streamlit Dashboard

The dashboard provides run controls, stage boards, logs, resource samples, risk indicators, model comparison charts, system maps, and report download sections. It is the user-facing visual layer for defence. It should be described as the operator dashboard rather than as the machine learning model itself.

The Streamlit dashboard is the examiner-facing and operator-facing layer of the prototype. It brings together controls, stage indicators, logs, metrics, risk cards, charts, reports, and system maps in one interface. This supports the study objective of representing the prediction workflow visually, not merely printing a model output in the terminal.

The dashboard was designed to make evidence visible. A user can see the selected scenario, the

current stage, the risk level, recent alerts, and report outputs. This is important because trust in a failure-prediction tool depends on whether the operator can understand why the system is flagging a run as risky.

3.9.5 Airflow DAG

The Airflow DAG provides orchestration evidence. It defines `prepare_run`, `run_pipeline_with_predictor`, `evaluate_intervention`, and `publish_report` tasks. The DAG shows how the predictor can sit inside a task workflow, but it is not deployed as part of the Render Streamlit service.

The Airflow DAG was included to show how the monitoring process can be placed inside an orchestrated workflow. The DAG structure makes the sequence explicit: prepare a run, execute the pipeline with the predictor, evaluate the intervention decision, and publish the report. This supports the claim that the prototype can be represented as a workflow rather than as an isolated script.

The DAG is also carefully scoped. It is evidence of orchestration design and local workflow integration, but it is not described as part of the Render web service. This distinction prevents deployment overclaiming while still showing that the project aligns with common pipeline orchestration practice.

3.9.6 Render Deployment Preparation

Render deployment is prepared for the Streamlit dashboard. The repository includes `render.yaml`, `.python-version`, `requirements.txt`, trained models, small sample data, and output charts. Large raw datasets and local report drafts are excluded from deployment.

Render deployment preparation focused on the web-facing Streamlit application because that is the part of the system most suitable for a lightweight hosted demonstration. The repository includes the files needed for Render to install dependencies and launch the dashboard with the correct host and port configuration.

Large raw traces, local report drafts, and generated files were excluded from deployment because they increase repository size without improving the hosted demonstration. The deployed prototype is therefore intentionally lean: it uses saved model artifacts, sample data, and output charts to demonstrate the workflow while keeping heavy retraining tasks local.

3.10 Risk Classification and Intervention Logic

Risk classification is based on two threshold values defined in `config.py`. A probability greater than or equal to 0.70 is classified as HIGH. A probability greater than or equal to 0.40 and below 0.70 is classified as MEDIUM. Values below 0.40 are classified as LOW. These thresholds provide a simple and explainable risk scale for the dashboard and report output.

The implementation also preserves raw model probability when demonstration calibration is applied. In `pipeline_monitor.py`, calibrated outputs retain `raw_model_failure_probability` so that the dashboard can present varied integer percentages during a simulation without destroying the underlying model evidence. This distinction is necessary in the report because synthetic presentation values are not evaluation metrics.

HIGH risk does not always terminate the pipeline automatically. The plugin supports policy behavior. In advisory or review mode, HIGH risk prompts the user to inspect the process and choose whether to continue or stop. In auto-stop mode, the system can stop the process when configured to do so. This design is defensible because production environments often require human approval before terminating a business process.

The LOW, MEDIUM, and HIGH categories translate model probability into an operator-friendly scale. This is necessary because raw probabilities can be difficult to interpret quickly during monitoring. The categories provide a simple risk language that can be used in the dashboard, report, and intervention decision.

The threshold values are intentionally transparent. A HIGH threshold of 0.70 and a MEDIUM threshold of 0.40 make the logic easy to explain during defence. However, the report also recognizes that production thresholds should be calibrated using operational cost assumptions. A missed failure and a false alarm do not have equal consequences, so final thresholds should reflect the priorities of the deployment environment.

The intervention design also avoids an unsafe assumption that the model should always make the final operational decision. Review mode allows the system to flag HIGH risk while giving the operator control over whether to continue or stop. This reflects real data-engineering practice, where termination can have business consequences and may require human approval.

The risk-classification layer was necessary because raw probability values alone are not always

actionable. A dashboard user may not immediately know whether a probability of 0.61 should be ignored, watched, or escalated. By converting probability into LOW, MEDIUM, and HIGH categories, the system creates a common operational language that can be understood quickly during a run. This is especially useful in a defence demonstration because the examiner can follow how model output becomes a visible monitoring decision.

The threshold design also supports reviewability. Because the threshold values are stated explicitly, another researcher or engineer can adjust them and observe how the number of warnings changes. In a production setting, these thresholds should be tuned using historical incident cost, expected downtime, and tolerance for false alarms. The present project uses explainable thresholds to support demonstration and evaluation, while recognizing that production calibration would require broader operational data.

The intervention logic separates prediction from action. A HIGH-risk classification identifies a state that requires attention, but the policy configuration decides whether the system advises, pauses for review, or stops the process. This separation is important because different organizations may have different risk tolerance. A financial reporting pipeline may require cautious stopping, while a low-impact experimental pipeline may prefer review before interruption.

3.11 Method of Data Analysis

Data analysis combines model-performance metrics with system-functionality evidence. Model-performance evidence is drawn from saved metrics and comparison files. System-functionality evidence is drawn from working code paths: the live runner, ETL pipeline, plugin, dashboard, report generator, and Airflow DAG.

Accuracy is reported, but it is not treated as sufficient for failure prediction. Failures are often less common than successful jobs, so accuracy can be high even when a model misses failures. Precision, recall, F1-score, PR-AUC, and confusion-matrix counts provide a better view of failure-class behavior.

Data analysis was performed at two levels. The first level was model analysis, where held-out metrics and comparison outputs were interpreted. The second level was system analysis, where implemented behavior was assessed through the dashboard, runner, plugin, report generator, and

Airflow DAG. Both levels were necessary because a strong metric alone would not prove that the monitoring workflow works.

The analysis also distinguishes controlled and production-style evidence. The ETL/HDFS-compatible result is expected to be stronger because the demonstration scenarios are structured and repeatable. The Alibaba and Google results are more difficult because they are heterogeneous, imbalanced, and source-specific. This distinction is carried into Chapter Four so that the findings are not overstated.

The output artifacts were used as evidence rather than decoration. Metrics files, charts, generated reports, and dashboard outputs are all connected to implemented code paths. This improves the credibility of the analysis because the results are traceable to stored project files.

3.12 Validity, Reliability and Reproducibility

Validity was supported by retaining source-specific schemas and avoiding a false merged-model claim. Reliability was supported by modular code structure. Reproducibility was supported by saved model artifacts, small deployable sample data, stored comparison outputs, and documented run commands.

The implementation has known boundaries. The ETL demonstration is controlled. The Alibaba and Google samples are bounded. The dashboard can use presentation calibration for demonstration. Airflow is included as orchestration evidence but is not started inside the Render web service. These boundaries are described openly so that the project remains academically defensible.

Internal validity was supported by avoiding label leakage, retaining source-specific schemas, and comparing the selected model against alternatives. These steps reduce the risk that the results are caused by an artificial preprocessing shortcut or by an unexamined model choice. The project also reports limitations openly where evidence is bounded or demonstrative.

Reliability was supported through modular implementation. Data loading, feature extraction, model training, comparison, live running, prediction, reporting, and dashboard rendering are handled by separate modules. This makes the system easier to test and revise because changes in one part of the system do not need to rewrite the entire prototype.

Reproducibility was supported by saving model artifacts, metrics, charts, sample data, and

configuration files. A future reviewer can inspect the files used in the dashboard and compare them with the methodology described in the report. Although full-trace retraining remains outside the deployable demonstration, the bounded experiments and saved artifacts provide a reproducible basis for the final-year project.

Construct validity was supported by aligning the features with recognizable operational warning signs. Severity counts, retry indicators, exception keywords, timing features, resource signals, and failed-task counts are meaningful because they describe conditions that engineers can understand. This does not prove root cause automatically, but it makes the model inputs easier to defend than arbitrary or hidden variables.

External validity remains limited because the live ETL demonstration is controlled and the public trace experiments are bounded. The study therefore avoids claiming that the model will generalize to every pipeline without retraining or recalibration. The correct claim is that the implemented approach demonstrates a workable failure-prediction and monitoring architecture that can be extended to new logs and larger traces.

Reliability was further supported by preserving the distinction between training artifacts and presentation artifacts. The dashboard may display rounded or calibrated values for readability during a demonstration, but the raw model probability and saved evaluation metrics remain available for academic interpretation. This distinction prevents the visual layer from being mistaken for the measured evidence.

3.13 Ethical Considerations

The project uses public traces and synthetic demonstration data. It does not collect human participant data. A future production deployment would require log privacy review because application logs can contain identifiers, credentials, or business-sensitive records. The prototype avoids overstating production readiness and distinguishes between measured model outputs and demonstration-layer presentation behavior.

The study does not use human participant data, but ethical considerations still apply because operational logs can contain sensitive information in real deployments. A production version would need procedures for masking credentials, limiting access to reports, protecting identifiers, and controlling who can view or export logs. The prototype avoids these risks by using public

traces and synthetic demonstration data.

The report also treats model limitations as an ethical issue. A failure-prediction system can influence whether a pipeline is stopped, delayed, or escalated. For this reason, the project avoids claiming that the model is production-ready without broader validation. It presents the system as a decision-support prototype that should assist human judgement rather than replace it.

3.14 Summary

This chapter describes the research design, datasets, sampling decisions, tools, preprocessing, feature engineering, model development, system architecture, risk policy, validation, reproducibility, and ethical considerations. The next chapter presents the implemented system and the stored evaluation evidence.

In summary, the methodology connects supervised learning, prototype development, source-specific modeling, live ETL monitoring, dashboard visualization, report generation, and orchestration evidence. The chapter also explains why the system uses bounded samples, why the public traces are not merged into one universal model, and why the dashboard separates demonstration behavior from measured evaluation evidence. This provides the foundation for the result presentation in Chapter Four.

CHAPTER FOUR

DATA PRESENTATION AND ANALYSIS

4.1 Preamble

This chapter presents the implemented system and the stored results produced by the project. The analysis is limited to available artifacts and implemented behavior. It does not claim external production deployment, full-trace retraining, or untested Airflow runtime behavior beyond the implemented DAG.

The chapter presents the results in a way that separates implementation evidence from model-performance evidence. Implementation evidence shows whether the software functions as a monitoring workflow. Model-performance evidence shows how the trained classifiers behaved on the available evaluation data. This separation is important because a dashboard may look complete even when a model is weak, and a model may score well without being integrated into a usable system.

The analysis also respects the limits of each evidence source. The controlled ETL experiment is useful for demonstrating the complete monitoring flow, while the Alibaba and Google samples are useful for showing how the approach behaves on more heterogeneous workload evidence. The results are therefore interpreted as prototype and benchmark evidence, not as a claim of enterprise-wide production validation.

4.2 Implemented System Output

The completed prototype provides three demonstration paths. The first is the synthetic simulation path, which generates controlled HDFS-style events. The second is the monitored ETL path, which runs the extract, validate, transform, load, and report process. The third is the attached

command path, where LivePipelineRunner executes an external command and streams its output through the predictor plugin.

The dashboard uses these paths to show the user how a data pipeline can be observed during execution. It presents the current stage, recent log lines, risk counts, current risk level, risk explanations, resource samples, intervention status, comparison charts, and reports. This directly addresses the earlier weakness of having only terminal output.

Table 4.1: Implemented demonstration modes

Mode	Code path	Purpose	Evidence
Synthetic simulation	simulation_engine.py	Controlled event stream for dashboard behavior.	Scenario selection, stage board, varied risk states.
Monitored ETL	etl_pipeline_demo.py	Realistic CSV pipeline with extract, validate, transform, load, and report stages.	Live ETL dashboard and report output.
Attached command	live_pipeline_runner.py	Plugin-style attachment to an external command.	Raw and normalized logs; resource samples.
Airflow orchestration	dags/pipeline_failure_monitor_demo.py	Workflow evidence for ordered tasks and intervention decision.	DAG tasks and configurable parameters.

The implemented system output demonstrates that the project moved beyond static prediction. A user can start a run, select a scenario, observe stage movement, inspect logs, watch risk counts change, review HIGH-risk events, and generate a report. These actions show that the model is embedded in a workflow that resembles how an operator might inspect a batch job.

The three demonstration paths also serve different evaluation purposes. Synthetic simulation tests whether the dashboard and risk categories respond to controlled event streams. The monitored ETL path tests whether a real process can produce logs and reports while the plugin observes it. The attached command path tests whether the monitoring idea can be generalized to an external command. Together, they support the claim that the predictor is not locked to a single script.

The output is especially useful for defence because it allows the examiner to see cause and effect. When warning or error messages increase, the feature extractor updates the evidence, the risk level changes, and the dashboard presents the state to the user. This makes the project easier to explain than an isolated metrics table because the relationship between pipeline behavior and prediction output is visible.

4.3 Model Evaluation Results

The main stored ETL/HDFS-compatible metrics show an accuracy of 0.9250, precision of 0.8421, recall of 1.0000, F1-score of 0.9143, ROC-AUC of 0.9883, and PR-AUC of 0.9704. These results support the controlled live demonstration, but they should not be generalized to all production pipelines without additional validation.

The Alibaba and Google 2019 benchmark results are more modest. This is expected because production-style workload traces are more heterogeneous and imbalanced. The report therefore presents these results as benchmark evidence rather than as proof that the live ETL model generalizes universally.

Table 4.2: Held-out XGBoost evaluation results preserved from project artifacts

Metric	ETL/HDFS-compatible model	Alibaba PAI sample	Google 2019 sample
Accuracy	0.9250	0.8768	0.6647
Precision	0.8421	0.2244	0.3258
Recall	1.0000	0.5147	0.5733
F1-score	0.9143	0.3125	0.4155
ROC-AUC	0.9883	0.7267	0.6709
PR-AUC	0.9704	0.3415	0.3057
MCC	0.8584	0.2830	0.2207
Specificity	0.8750	0.8976	0.6887
Log loss	0.1222	0.4328	0.7793
Confusion matrix	TP 16; TN 21; FP 3; FN 0	TP 35; TN 1,061; FP 121; FN 33	TP 301; TN 1,378; FP 623; FN 224

Table 4.3: Classifier comparison for the ETL/HDFS-compatible experiment

Algorithm	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.992	0.963	0.963	0.963	0.9977
HistGradientBoosting	0.992	1.000	0.9259	0.9615	0.9887
XGBoost	0.988	1.000	0.8889	0.9412	0.9894
Random Forest	0.980	0.9231	0.8889	0.9057	0.9475
Decision Tree	0.976	0.8889	0.8889	0.8889	0.9437
Majority Baseline	0.892	0.000	0.000	0.000	0.5000

Table 4.4: Classifier comparison for bounded Alibaba and Google benchmark samples

Dataset	Best F1 model	Best F1	XGBoost F1	Interpretation
Alibaba PAI sample	HistGradientBoosting	0.4324	0.3238	The production-trace sample is

Dataset	Best F1 model	Best F1	XGBoost F1	Interpretation
Google 2019 sample	Random Forest	0.4409	0.3234	imbalanced and harder than the controlled ETL demonstration. The random forest comparison produced stronger F1 than XGBoost on this bounded sample.

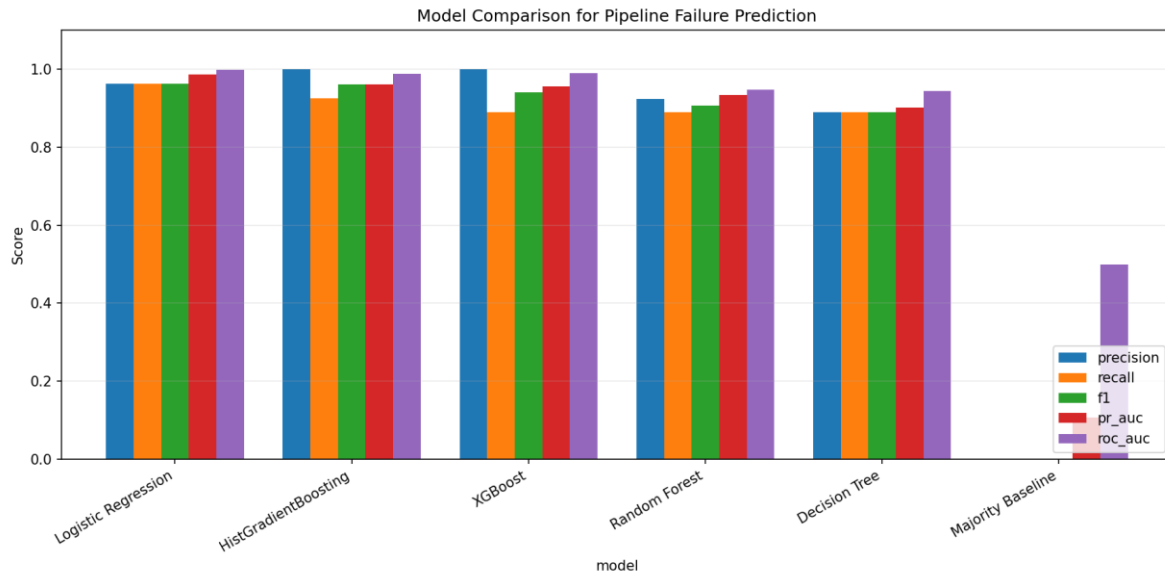


Figure 4.1: Model comparison chart for ETL/HDFS-compatible experiment

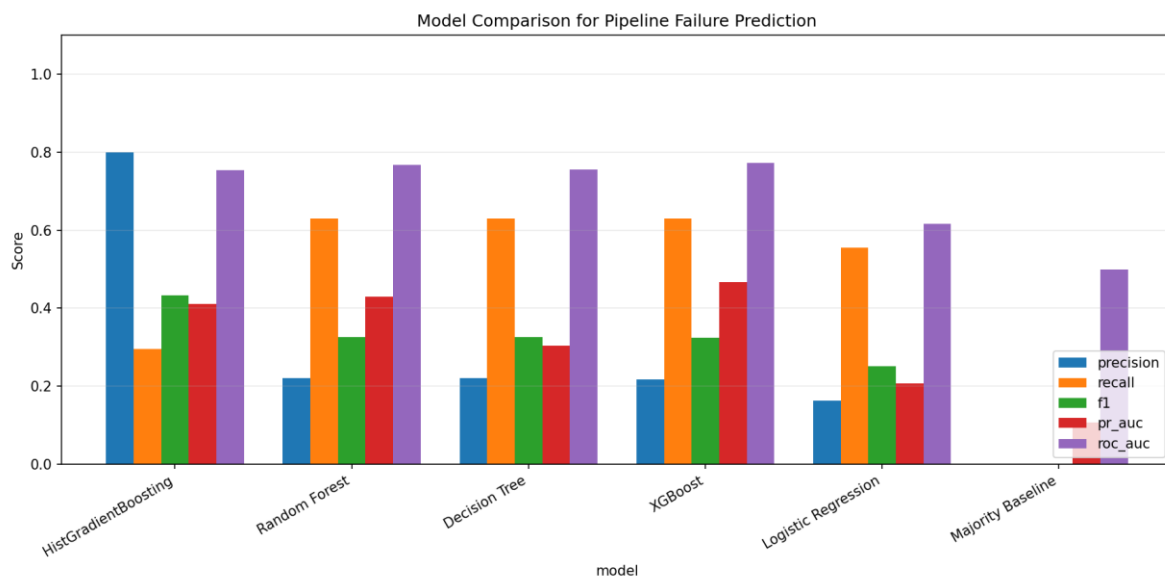


Figure 4.2: Model comparison chart for bounded Alibaba PAI sample

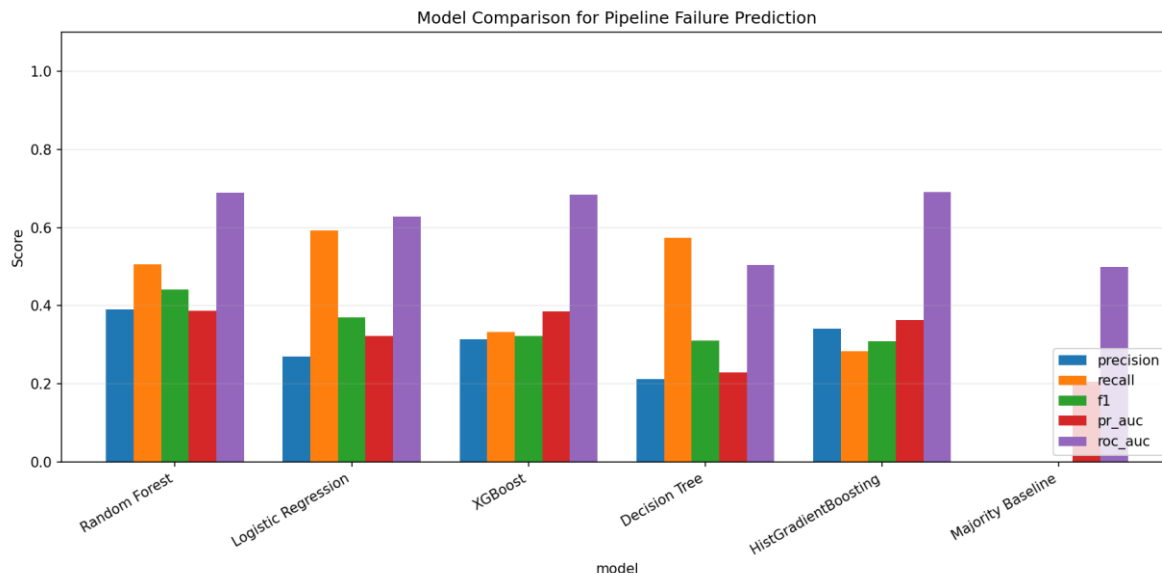


Figure 4.3: Model comparison chart for bounded Google 2019 sample

The ETL/HDFS-compatible model achieved strong held-out results, especially on recall and F1-score. High recall is valuable in failure prediction because missed failures can be more damaging than false alarms. However, high recall must still be considered alongside precision because too many false alarms would reduce operator trust. The reported F1-score therefore provides a more balanced view than accuracy alone.

The Alibaba and Google benchmark results provide a more conservative view of the model family. Their lower scores show that production-style workload traces are more difficult than a controlled ETL demonstration. This is an important finding because it prevents the report from presenting the strongest result as if it automatically applies to every environment.

The classifier-comparison evidence also shows that the best model may differ by source. XGBoost was strong for the implemented live demonstration, but histogram gradient boosting and random forest performed better on the bounded Alibaba and Google comparisons respectively. This supports a data-source-specific model-selection strategy rather than a rigid rule that one algorithm should always be used.

The strong recall recorded for the ETL/HDFS-compatible model is meaningful because the project is concerned with early recognition of risky execution. In a monitoring environment, missing a failure can be more serious than raising a warning that later proves unnecessary. However, recall must not be interpreted alone. Precision is also necessary because repeated false

alarms can make operators ignore the system. The F1-score therefore provides a balanced measure of the failure-class result.

The ROC-AUC and PR-AUC values add another layer to the interpretation. ROC-AUC describes the ability of the model to rank positive and negative cases across thresholds, while PR-AUC is particularly useful when the failure class is less common. This is why PR-AUC was retained in the project artifacts. It helps show whether the model remains useful when attention is focused on the minority failure class rather than the majority successful class.

The Alibaba and Google results are important precisely because they are not as strong as the controlled ETL result. They show that workload traces contain more variation and that the model may need source-specific tuning. This finding strengthens the report because it demonstrates that the project does not hide difficult evidence. Instead, it uses the benchmark results to identify realistic limits and future improvement areas.

4.4 Interpretation of the Evaluation

The ETL/HDFS-compatible model has the strongest scores because the demonstration data is controlled and designed around observable stages. This makes it suitable for a defence demonstration in which the aim is to show the end-to-end monitoring concept. However, this result should be presented with the limitation that controlled scenarios are easier than open production traces.

The Alibaba result shows why accuracy alone can mislead. Class imbalance means that a model can correctly identify many successful jobs while still struggling to detect failures. The Google 2019 result also shows that different datasets may prefer different model families. Random forest produced the best F1-score in the Google comparison, while histogram gradient boosting produced the best F1-score in the Alibaba comparison.

The correct conclusion is therefore balanced. The project implements XGBoost as the selected predictor and demonstrates it in the pipeline dashboard, but the comparison evidence shows that model selection should remain data-source-specific when the system is extended.

The results should be interpreted through the purpose of the project. The objective was to build a working prototype that predicts and visualizes batch-job failure risk, not to publish a universal model for all cluster workloads. From that perspective, the ETL/HDFS-compatible result shows

that the implemented live demonstration is technically consistent, while the benchmark results show the need for further validation before production use.

The controlled ETL result is valuable because it proves the end-to-end path from logs to features, prediction, risk classification, review controls, and report generation. At the same time, the bounded production-trace results remind the reader that more diverse environments can reduce model performance. The correct interpretation is therefore constructive rather than exaggerated: the prototype works, but broader validation remains necessary.

The comparison also highlights the danger of relying only on accuracy. In imbalanced failure-prediction problems, a model can classify many successful jobs correctly and still miss failures. For that reason, recall, precision, F1-score, PR-AUC, and MCC are more relevant to the failure-class question. This supports the metric choices described in the methodology.

The evaluation also shows the difference between demonstration validity and generalization validity. Demonstration validity asks whether the implemented system can run, observe logs, classify risk, present evidence, and generate reports. Generalization validity asks whether the same model will perform well on a broader set of production pipelines. The project provides stronger evidence for the first question and preliminary benchmark evidence for the second.

This interpretation is consistent with the selected research design. The prototype is intended to show an end-to-end monitoring concept for batch jobs using machine learning, not to replace enterprise observability platforms. The results therefore support the claim that machine learning can be integrated into a visual pipeline-monitoring workflow. They do not support a claim that the current thresholds or model files should be used unchanged in every production environment.

4.5 Dashboard Analysis

The dashboard is organized around run control, live monitoring, logs, resources or metrics, model comparison, reports, and system view. These tabs make the workflow understandable to a supervisor or examiner because the user can see not only the final prediction but also the pipeline stages, logs, risk evidence, and report output.

The sidebar lets the user choose demo mode, scenario, minimum lines before prediction, policy behavior, and playback settings. In attached command mode, the user can provide a command that the runner executes. The dashboard therefore represents the predictor plugin as something

that can work beside a pipeline, not only as a static classifier.

The HIGH-risk review panel is one of the most important defence features. When HIGH risk is detected in review mode, the dashboard exposes the flagged process and allows the operator to continue or stop. This reflects the project requirement that HIGH risk should not silently pause the pipeline without giving the user a meaningful decision.

Table 4.5: Dashboard components and evidence value

Component	What it displays	Why it matters
Run Control	Mode, scenario, policy, seed, loop and command options.	Shows that the demo is configurable.
Stage Board	Extract, validate, transform, load and report stages.	Makes the pipeline visually recognizable.
Log Stream	Recent generated or attached log lines.	Shows evidence behind predictions.
Risk KPIs	Current risk, counts, HIGH-risk status and explanations.	Connects model output to operator decision-making.
Review Controls	Continue and stop options for HIGH risk.	Demonstrates intervention workflow.
Model Comparison	Saved charts and comparison tables.	Supports the algorithm comparison objective.
Report Tab	JSON payload and generated report paths.	Supports post-run auditability.
System Map	High-level architecture of modules.	Helps explain the implementation during defence.

The dashboard provides the clearest evidence that the project has an operator-facing workflow. The run-control area allows the user to select a mode and scenario, while the monitoring panels show the current state of the process. This makes the system interactive and inspectable, which is important for a final-year defence because the examiner can follow the pipeline as it moves through its stages.

The log stream and stage board help connect the prediction to observable behavior. A HIGH-risk state is more understandable when it appears beside warning lines, failed validation messages, retry attempts, or stage delays. This reduces the black-box feeling of the model because the operator can see the operational signals surrounding the prediction.

The review controls are also a significant result. They show that the project does not treat model output as an automatic command in every case. Instead, the system can pause for human review, allow continuation, or stop according to policy. This is important in real operations because stopping a pipeline can protect data quality but may also delay downstream business processes.

The model-comparison and report sections extend the dashboard beyond live viewing. They allow the user to inspect stored evidence and export a monitoring report. This supports

accountability because the session can be reviewed after the dashboard run ends.

4.6 Airflow Orchestration Evidence

Airflow is used as orchestration evidence rather than as the primary dashboard. The DAG named `pipeline_failure_monitor_demo` has no schedule by default, which makes it suitable for manual defence triggering. It includes parameters for `demo_type`, `pipeline_command`, `scenario`, `loops`, `min_lines`, `stop_policy`, and `fail_airflow_task_on_high`.

The DAG tasks are `prepare_run`, `run_pipeline_with_predictor`, `evaluate_intervention`, and `publish_report`. This sequence proves that the project workflow can be represented as ordered pipeline tasks. The `evaluate_intervention` task checks HIGH-risk counts and can raise an Airflow exception if configured to fail the orchestration task on HIGH risk.

This evidence should be described carefully. The repository contains the Airflow DAG and the DAG calls the same project engines, but Render deployment does not start an Airflow server. Airflow remains part of the project evidence and local orchestration demonstration.

The Airflow DAG demonstrates that the monitoring logic can be represented as a sequence of workflow tasks. This is important because data pipelines are often orchestrated rather than executed as one manual command. By defining preparation, monitored execution, intervention evaluation, and report publishing as tasks, the project shows how the predictor can fit into a broader workflow model.

The DAG parameters also support experimentation. Values such as demo type, command, scenario, loop count, minimum lines, stop policy, and failure-on-high behavior can be configured for different runs. This gives the workflow flexibility and makes the orchestration evidence more than a static file.

The interpretation remains deliberately cautious. The DAG is included as local orchestration evidence, while the Render deployment focuses on the Streamlit dashboard. This distinction is important because deploying Airflow as a production scheduler would require additional infrastructure, security, persistence, and worker configuration.

4.7 Report Generation Evidence

The report generator creates JSON and HTML reports. The JSON report preserves structured

monitoring evidence, while the HTML report presents operator-facing summary cards, risk counts, high-risk entries, alert rows, log rows, and resource summaries. This improves the project because monitoring does not end when the visual session stops.

The predictor plugin report payload includes session status, stop policy, current risk, risk counts, alerts, high-risk items, logs, stage summary, resource summary, and decision metadata. This structure supports auditability and gives the examiner a way to inspect what happened after a run.

Table 4.6: Report payload evidence

Report element	Evidence recorded	Usefulness
Session summary	Status, current risk, counts and policy.	Explains the run outcome.
Alerts	Risk level, probability, explanation and timestamp.	Preserves prediction history.
High-risk items	Filtered HIGH-risk alerts.	Supports intervention review.
Logs	Recent emitted or adapted messages.	Links decisions to process evidence.
Resource summary	CPU, memory, disk and backend metadata.	Adds runtime context.
Report paths	JSON and HTML file locations.	Supports post-run inspection.

Report generation is a key part of the system because operational monitoring should leave a reviewable record. A dashboard session is temporary, but a JSON or HTML report can preserve what happened during the run. This supports accountability and makes the system more useful for post-run diagnosis.

The JSON report is valuable for structured inspection because it stores fields that can later be loaded by another tool or analyzed programmatically. The HTML report is valuable for human review because it presents session status, risk counts, high-risk items, logs, and resource summaries in a readable format. Together, they support both machine-readable and operator-readable evidence.

The report payload also connects the prediction to the intervention decision. When HIGH risk appears, the report can preserve the alert details and the policy context. This helps distinguish between a model score, an operator decision, and a final pipeline outcome.

4.8 Deployment Readiness Analysis

The deployment preparation focuses on the Streamlit dashboard. The repository now includes `render.yaml` and `.python-version`. The `Render start` command runs Streamlit on 0.0.0.0 and uses

the Render-provided port. This supports web deployment of the visual dashboard.

The deployment is intentionally lightweight. It excludes large raw datasets, installer files, generated reports, Word documents, and training logs. The deployed service uses trained model artifacts, small sample data, and saved comparison evidence. This is appropriate because a cloud demo should not require uploading gigabytes of raw traces.

The project should still be described as a prototype deployment. Render deployment proves the dashboard can be hosted as a web service, but it does not prove enterprise production readiness, continuous retraining, authentication, multi-user access control, or long-term persistent storage.

Deployment readiness was assessed from the perspective of a lightweight defence demonstration. The repository contains the files needed for a Streamlit service to run on Render, including dependency declarations, a Python version file, and a Render configuration. This means the visual dashboard can be hosted without requiring the examiner to install the full local environment.

The deployment decision is intentionally conservative. Raw public traces, generated reports, installer files, and local document drafts were excluded because they increase size and complexity. A deployable prototype should include only the artifacts required for demonstration: source code, model files, sample data, and saved charts. This conservative approach is also consistent with broader data-warehousing discussions that emphasize cloud readiness, scalability, and maintainable data architecture ([Srikanth & Vishnu, 2024](#)).

The project is not described as a complete enterprise deployment. Production readiness would require authentication, authorization, secret management, persistent storage, logging retention, model registry support, continuous monitoring, retry handling, and infrastructure-level observability. The current deployment evidence is therefore suitable for a final-year prototype but not for unsupported production use.

A useful part of deployment readiness is the separation between files needed for demonstration and files needed for research training. The hosted dashboard does not need raw multi-gigabyte trace archives to show the monitoring workflow. It needs the trained artifacts, sample data, configuration, and visual components. This distinction keeps the deployment manageable while still preserving the research explanation inside the report.

The deployment analysis also identifies what would be required beyond the final-year prototype. A production version would need secure handling of logs, authentication for users, persistent storage for reports, audit trails, monitoring of the monitoring service itself, and a policy for model updates. These requirements are deliberately stated so that the report does not confuse a successful hosted demo with a complete enterprise system.

4.9 Summary of Findings

The findings show that the project has progressed from a terminal-based predictor to a visual monitoring prototype with live ETL execution, plugin-style ingestion, risk classification, HIGH-risk intervention controls, report generation, model comparison evidence, and Airflow orchestration evidence. The strongest technical value is the end-to-end workflow rather than any single score. The main limitation is that production-scale validation remains future work.

The findings show that the project achieved a practical integration of model evidence and system behavior. The implemented workflow can run a pipeline, collect logs, update risk, show intervention controls, and generate reports. The evaluation results show strong controlled ETL performance and more modest benchmark performance, which together provide a balanced view of the prototype.

The strongest contribution is the end-to-end monitoring concept. Instead of presenting failure prediction as an isolated classifier, the project presents it as a plugin-style workflow that supports observation, decision-making, and auditability. The main weakness is that full production-scale validation and multi-user deployment remain outside the present scope.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

The project developed a log analysis and failure prediction system for data pipeline batch jobs using XGBoost and supporting machine learning comparisons. The system includes data loaders, feature extractors, model training, model comparison, a predictor plugin, a live ETL runner, a Streamlit dashboard, report generation, and an Airflow DAG.

The central improvement over the earlier implementation is the visual and operational workflow. The user can start a pipeline, observe stages and logs, view changing risk classifications, inspect HIGH-risk cases, continue or stop the process, and generate a report. This makes the project easier to defend because it demonstrates how the model would work beside a real pipeline process.

The study addressed the problem of detecting and presenting failure risk in data pipeline batch jobs. It combined machine-learning model development with a software prototype that can observe logs, extract features, classify risk, display evidence, and generate reports. This combination is important because failure prediction is useful only when the prediction reaches an operator in a form that can guide action.

The project also improved the defensibility of the work by separating evidence layers. Offline training and comparison outputs provide model evidence, the ETL demonstration provides

runtime evidence, the predictor plugin provides reusable monitoring behavior, the Streamlit dashboard provides visual evidence, and the Airflow DAG provides orchestration evidence. These layers make the project stronger than a single notebook or static interface.

The results show that the controlled ETL/HDFS-compatible model performed strongly, while the bounded Alibaba and Google benchmarks were more challenging. This balanced outcome is important because it shows that the prototype is promising but still requires broader validation before stronger generalization claims can be made.

5.2 Conclusion by Objectives

Table 5.1: Objective-level conclusion

Objective	Conclusion
Identify failure patterns	The feature engineering layer captures severity counts, failure keywords, retry behavior, task failures, scheduling delay and resource context.
Compare models	Saved comparison outputs show that XGBoost was evaluated beside logistic regression, decision tree, random forest, histogram gradient boosting and a baseline.
Develop monitoring predictor	The predictor plugin and PipelineMonitor connect trained artifacts to streaming logs and produce risk classifications.
Evaluate model	Stored metrics provide precision, recall, F1, ROC-AUC, PR-AUC and confusion-matrix evidence.
Visualize workflow	The Streamlit dashboard and Airflow DAG provide visual monitoring and orchestration evidence.

The first objective, which concerned the identification of failure patterns, was achieved through feature engineering. The system extracts severity counts, error and warning rates, failure keywords, retry evidence, timing features, and workload-structure features. These features make raw operational evidence usable for supervised learning and live monitoring.

The second objective, which concerned model comparison, was achieved through saved comparison outputs. The study evaluated multiple classifiers and retained the results so that the selected implementation could be discussed against alternatives. This is important because the best model was not assumed without comparison.

The third objective, which concerned the development of a monitoring predictor, was achieved through the predictor plugin, live runner, and dashboard integration. The system can receive log evidence, update risk classification, store alerts, and present the state to the user. The fourth objective, which concerned evaluation, was achieved using metrics such as precision, recall, F1-score, ROC-AUC, PR-AUC, MCC, specificity, and confusion-matrix counts. The fifth objective,

which concerned visualization, was achieved through the Streamlit dashboard and Airflow workflow evidence.

Overall, the objectives were achieved in a connected manner rather than as isolated tasks. Data preparation made feature extraction possible, feature extraction supported model training, model comparison justified the selected implementation, the predictor plugin connected the model to streaming logs, and the dashboard made the results visible. This chain of work is important because it shows that the project moved from data to decision support.

The objective related to visualization was particularly important for the final project because it transformed the work from a technical model into a demonstrable system. The dashboard, report output, and Airflow DAG make the system easier to inspect and defend. They show not only what the model predicted, but also how the prediction fits into a recognizable data pipeline workflow.

5.3 Recommendations

Keep the deployed Render service focused on the Streamlit dashboard and plugin demonstration. Airflow should remain orchestration evidence unless a separate Airflow deployment is required.

Add a visible dashboard toggle that separates raw model probability from demonstration presentation percentage. This will make the honesty of the demonstration even clearer.

Expand production-trace validation when more compute time is available. Larger Alibaba and Google samples should be evaluated in repeated runs before making stronger generalization claims.

Introduce model-version tracking so that each report can identify the model artifact used for a monitoring session.

Add authentication if the dashboard is used beyond defence demonstration, because pipeline logs and predictions can contain sensitive operational information.

Add explainability tools such as SHAP only after the existing model pipeline is stable, and label such explanations as model explanations rather than causal proof.

Calibrate LOW, MEDIUM and HIGH thresholds with user-defined cost assumptions before

production deployment. A missed failure and a false alarm do not have equal operational cost.

The first recommendation is to treat the current system as a validated prototype rather than a finished enterprise platform. It is suitable for demonstration, research discussion, and further development, but a production deployment should include stronger operational controls such as authentication, persistent storage, model registry support, and structured logging.

The second recommendation is to continue separating raw model probability from demonstration presentation values. This distinction protects the academic honesty of the work. The dashboard can remain visually clear during defence while the report still preserves the raw model evidence used for evaluation.

The third recommendation is to expand validation before deployment in a real organization. Larger and repeated experiments should be carried out on additional traces, real pipeline logs, and different failure categories. This will help determine whether the thresholds, features, and selected model remain effective outside the controlled demonstration.

The fourth recommendation is to improve explainability only after the base monitoring workflow remains stable. Methods such as SHAP can help show which features influenced a prediction, but explanations should be presented as model explanations rather than proof of the true root cause. Root-cause confirmation still requires engineering investigation.

The system should be extended with stronger report storage. At present, reports can be generated for inspection, but a production-oriented version should store them in a database with session identifiers, timestamps, model versions, risk counts, operator decisions, and final job outcomes. This would support trend analysis and make it possible to compare predictions against later confirmed failures.

Another recommendation is to introduce model-version governance. Each monitoring report should record the model name, training date, feature schema, threshold configuration, and evaluation summary used during that session. This will make future audits easier because an engineer can determine exactly which model produced a particular warning.

The dashboard should also include clearer separation between operational controls and research evidence. Operational users need simple risk cards, alerts, and actions, while researchers and

examiners may need metrics, comparison charts, and feature explanations. Separating these views would make the interface cleaner for each audience while preserving the evidence needed for academic review.

5.4 Limitations of the Study

The live ETL process is a controlled demonstration pipeline. It represents recognizable data engineering stages but does not reproduce all dependencies, storage systems, orchestration queues, or business constraints of an enterprise environment.

The Alibaba and Google experiments are bounded local samples. They are useful benchmark evidence, but they are not full-trace production validation. The report therefore avoids claiming that the model has been proven across all workloads in those traces.

The dashboard can produce varied integer risk percentages for presentation during synthetic demonstration. This behavior is useful for a defence session because it prevents the interface from appearing static, but those presentation values are not the same as raw model probabilities.

The Render deployment prepares the Streamlit dashboard as a web service. It does not deploy Airflow, background workers, persistent storage, user authentication, or continuous retraining.

One limitation is the controlled nature of the live ETL demonstration. Although it includes recognizable stages and realistic log messages, it does not include all the dependencies of a real enterprise pipeline, such as distributed storage, message queues, authentication systems, external service failures, or multiple simultaneous workloads. This limits the extent to which the live demonstration can be generalized.

Another limitation is the bounded size of the benchmark samples. The Alibaba and Google traces are useful because they broaden the evidence base, but the current experiments do not exhaust the full public datasets. Larger repeated experiments would be required to make stronger claims about general performance across those traces.

A further limitation is threshold calibration. The LOW, MEDIUM, and HIGH thresholds are explainable and useful for demonstration, but production environments should calibrate them using actual business costs. For example, the cost of missing a failure may be higher than the cost of investigating a false alarm in some systems, while the reverse may be true in others.

The deployment limitation is also important. Render deployment supports the Streamlit dashboard, but it does not provide a complete production architecture with Airflow workers, persistent databases, user access management, or continuous retraining. The hosted version should therefore be understood as a web demonstration of the prototype.

The study is also limited by the absence of long-term monitoring history. The prototype can demonstrate a session and generate a report, but it does not yet analyze patterns across weeks or months of repeated pipeline execution. Long-term history would be necessary to identify seasonal failure trends, recurring dependency problems, model drift, and the effect of threshold changes over time.

Another limitation is that the system does not yet include formal explainability in the live dashboard. The risk explanations are based on feature evidence and alert summaries, but methods such as SHAP were not integrated into the final deployed workflow. This limits the depth of model interpretation available to the operator, although it also keeps the prototype simpler and easier to defend.

A final limitation concerns operational integration. The live runner demonstrates attachment to an external command, and the Airflow DAG shows orchestration evidence, but the system has not been integrated with enterprise schedulers, data warehouses, incident-management tools, or notification channels. Such integration would be required before the system could support a real operations team.

5.5 Suggestions for Further Work

Future work should package the predictor as a formal plugin with a documented API, configuration file, and installable distribution. This would make it easier for another pipeline to call the monitor without copying project scripts.

A future version should add persistent storage for monitoring sessions. Storing reports in a database would support historical trend analysis, threshold tuning, and repeated defence of model decisions.

Another extension is to support multiple simultaneous pipelines. The current dashboard is strongest as a single-session demonstration; a production operations console would need session isolation, user permissions, queueing, and alert routing.

Further research could compare additional sequence-aware log models against the current engineered-feature approach. Such models should be evaluated carefully because higher complexity does not automatically produce better operational decisions.

Future work should evaluate the system with real organizational pipeline logs after privacy and access controls have been addressed. Such evaluation would help confirm whether the feature engineering approach remains useful when logs include inconsistent message formats, missing fields, operational noise, and business-specific failure modes.

Another direction is to package the predictor as a formally documented plugin or service. A clear API, configuration file, installable package, and versioned model artifacts would make it easier to connect the monitor to other pipelines without copying project scripts manually.

Future work should also add persistent monitoring history. Storing alerts, decisions, reports, and outcomes in a database would allow trend analysis, threshold tuning, model drift detection, and post-incident review. This would move the prototype closer to an operations console.

A final direction is to compare engineered-feature methods with sequence-aware log models. Deep learning approaches such as log-sequence anomaly detection can capture order and context in log streams ([Du et al., 2017](#)). However, future research should compare such methods carefully because greater complexity must be justified by better operational performance, interpretability, and maintainability.

5.6 Final Conclusion

The project provides a credible final-year prototype for log analysis and failure prediction in data pipeline batch jobs. Its strongest contribution is the integration of machine learning evidence with an understandable operational workflow: a pipeline runs, logs are observed, features are extracted, failure risk is classified, HIGH risk triggers review, and a report is generated. The implementation is not presented as a finished production platform, but it is a defensible foundation for a monitoring plugin that can be extended and evaluated further.

The final conclusion is that the project successfully demonstrates a defensible prototype for log analysis and failure prediction in data pipeline batch jobs. It combines supervised learning, source-specific model artifacts, live ETL monitoring, dashboard visualization, HIGH-risk review logic, report generation, and orchestration evidence. This combination directly supports the project aim because it shows not only that failure risk can be estimated, but also that the estimate can be presented within a practical monitoring workflow.

The project should be understood as a foundation for further development. Its strongest academic value is the clear integration of model evidence and operational workflow, while its main practical limitation is the need for broader validation and production hardening. With larger datasets, calibrated thresholds, persistent storage, authentication, and extended plugin packaging, the system could become a stronger tool for proactive data-pipeline reliability management.

The completed work also shows that a final-year project can combine research evidence with a functioning software artifact. The literature review explains why pipeline failure prediction matters, the methodology explains how data and models were handled, the results chapter presents the implemented workflow and evaluation outputs, and the conclusion identifies the boundaries of the prototype. This progression makes the project coherent because every chapter supports the same central idea: pipeline failures should be detected early and presented in a way that supports action.

The most important lesson from the study is that failure prediction should not be treated as a single model score. A useful monitoring system must connect data preparation, feature engineering, model evaluation, live observation, risk interpretation, intervention policy, and reporting. The implemented prototype brings these parts together in a clear and defensible manner. Although further validation and production hardening are still required, the work provides a strong basis for future research and practical development in intelligent data pipeline monitoring.

The project is also valuable because it makes the boundary between automation and human judgement visible. The model estimates failure risk, but the operator still needs supporting evidence before making a decision. By showing logs, stages, alerts, resource samples, and reports together, the system encourages informed review rather than blind trust in automation. This is an important principle for data engineering because stopping a pipeline, allowing it to

continue, or escalating it to another engineer can affect reporting schedules, business users, and downstream systems.

In practical terms, the prototype demonstrates a path for improving batch-job reliability without requiring an organization to discard its existing pipelines. The live attachment design shows that a predictor can observe an external process, translate the evidence into structured features, classify the risk, and preserve the session for later inspection. This makes the work extensible. Future versions can replace the sample ETL process with organization-specific commands, connect reports to persistent storage, and tune thresholds using real incident history.

For academic purposes, the study remains defensible because it states both its contribution and its limits. The contribution is the integration of predictive modeling with a visible monitoring and intervention workflow. The limit is that broader validation, stronger security controls, persistent storage, and enterprise integration are still required before production use. This balanced conclusion protects the report from overclaiming while still showing that the implemented system satisfies the core objectives of the final-year project.

REFERENCES

- Alibaba Group. (2022). *Cluster trace GPU v2020* [Data set]. GitHub.
<https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2020>
- Akash, V. C., & Charate, P. C. (2025). Proactive data pipeline maintenance via machine learning-driven anomaly detection. *International Journal of Scientific Research in Science and Technology*, 12(2), 1041-1053. <https://doi.org/10.32628/ijrst251222663>
- Apache Software Foundation. (n.d.). *DAGs*. Apache Airflow documentation. Retrieved June 2, 2026, from <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>
- Arif, A. Z., Abduljabbar, H. Z., Tahir, A. H., Bibo, S. A., & Almufti, S. M. (2023). eXtreme gradient boosting algorithm with machine learning: A review. *Academic Journal of Nawroz University*, 12(2), 320-334. <https://doi.org/10.25007/ajnu.v12n2a1612>

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
<https://doi.org/10.1023/A:1010933404324>
- Chanda, D. (2024). Automated ETL pipelines for modern data warehousing: Architectures, challenges, and emerging solutions. *The Eastasouth Journal of Information System and Computer Science*, 1(3), 209-212. <https://doi.org/10.58812/esiscs.v1i03>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785-794). <https://doi.org/10.1145/2939672.2939785>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.
<https://doi.org/10.1007/BF00994018>
- Du, M., Li, F., Zheng, G., & Srikumar, V. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (pp. 1285-1298).
<https://doi.org/10.1145/3133956.3134015>
- Foidl, H., Golendukhina, V., Ramler, R., & Felderer, M. (2024). Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers. *Journal of Systems and Software*, 207, Article 111855.
<https://doi.org/10.1016/j.jss.2023.111855>
- Google. (n.d.). *Borg cluster workload traces from 2019* [Data set documentation]. GitHub. Retrieved June 2, 2026, from <https://github.com/google/cluster-data/blob/master/ClusterData2019.md>
- Informatica. (2025). *What is an ETL pipeline?*

<https://www.informatica.com/resources/articles/what-is-etl-pipeline.html>

Jassas, M. S., & Mahmoud, Q. H. (2022). Analysis of job failure and prediction model for cloud computing using machine learning. *Sensors*, 22(5), Article 2035. <https://doi.org/10.3390/s22052035>

Maalouf, M. (2011). Logistic regression in data analysis: An overview. *International Journal of Data Analysis Techniques and Strategy*, 3(3), 281-299.

Matzavela, V., & Alepis, E. (2021). Decision tree learning through a predictive model for student academic performance in intelligent m-learning environments. *Computers and Education: Artificial Intelligence*, 2, Article 100035. <https://doi.org/10.1016/j.caeai.2021.100035>

Mienye, I. D., & Jere, N. (2024). A survey of decision trees: Concepts, algorithms, and applications. *IEEE Access*, 12, 86716-86727. <https://doi.org/10.1109/ACCESS.2024.3416838>

Naeem, M., Rizvi, S., & Coronato, A. (2020). A gentle introduction to reinforcement learning and its application in different fields. *IEEE Access*, 8, 209320-209344. <https://doi.org/10.1109/ACCESS.2020.3038605>

Naeem, S., Ali, A., Anam, S., & Ahmed, M. M. (2023). Unsupervised machine learning algorithms: Comprehensive review. *International Journal of Computing and Digital Systems*, 13(1), 911-921. <https://doi.org/10.12785/ijcds/130172>

Nishanth R. M. (2019). The evolution of ETL architecture: From traditional data warehousing to real-time data integration. *World Journal of Advanced Research and Reviews*, 1(3), 073-084. <https://doi.org/10.30574/wjarr.2019.1.3.0033>

- Ogunsola, K. O., Balogun, E. D., & Ogunmokun, A. S. (2022). Developing an automated ETL pipeline model for enhanced data quality and governance in analytics. *International Journal of Multidisciplinary Research and Growth Evaluation*, 3(1), 791-796. <https://doi.org/10.54660/ijmrge.2022.3.1.791-796>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://www.jmlr.org/papers/v12/pedregosa11a.html>
- Popovic, A., Ivkovic, V., Trajkovic, N., & Lukovic, I. (2024). A domain-specific language for managing ETL processes. *PeerJ Computer Science*, 10, Article e1835. <https://doi.org/10.7717/peerj-cs.1835>
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106. <https://doi.org/10.1007/BF00116251>
- Reddy, P., Viswanath, P., & Reddy, B. E. (2018). Semi-supervised learning: A brief review. *International Journal of Engineering & Technology*, 7(1), 81-85.
- scikit-learn developers. (n.d.-a). *f1_score*. Scikit-learn documentation. Retrieved June 2, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html
- scikit-learn developers. (n.d.-b). *StratifiedGroupKFold*. Scikit-learn documentation. Retrieved June 2, 2026, from https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedGroupKFold.html
- Seenivasan, D. (2025). AI-driven enhancement of ETL workflows for scalable and efficient

- cloud data engineering. *International Journal of Engineering and Computer Science*, 13(20), 26837-26848. <https://doi.org/10.18535/ijecs/v14i02.4824>
- Shaveta. (2023). A review on machine learning. *International Journal of Science and Research Archive*, 9(1), 281-285. <https://doi.org/10.30574/ijrsra.2023.9.1.0410>
- Srikanth, G., & Vishnu, C. (2024). The future of data warehousing: Trends, technologies, and challenges in the era of big data, cloud computing, and artificial intelligence. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, 10(5), 470-479. <https://doi.org/10.32628/cseit241051029>
- Streamlit. (n.d.). *st.fragment*. Streamlit documentation. Retrieved June 2, 2026, from <https://docs.streamlit.io/develop/api-reference/execution-flow/st.fragment>
- Suyal, M., & Goyal, P. (2022). A review on analysis of k-nearest neighbor classification machine learning algorithms based on supervised learning. *International Journal of Engineering Trends and Technology*, 70(7), 43-48. <https://doi.org/10.14445/22315381/IJETT-V70I7P205>
- Udousoro, I. C. (2020). Machine learning: A review. *Semiconductor Science and Information Devices*, 2(2), 5-14. <https://doi.org/10.30564/ssid.v2i2.1931>
- Weng, Q., Xiao, L., Yu, C., Wang, W., Wang, C., He, J., Li, Y., Zhang, L., & Ding, W. (2022). MLaaS in the wild: Workload analysis and scheduling in large-scale heterogeneous GPU clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)* (pp. 945-960). <https://www.usenix.org/conference/nsdi22/presentation/weng>
- Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X., & Zhang, W. (2021). Semi-supervised log-based anomaly detection via probabilistic label estimation. In *Proceedings*

- of the International Conference on Software Engineering* (pp. 1448-1460).
<https://doi.org/10.1109/ICSE43902.2021.00130>
- Yuan, Y., Wu, Y., Wang, Q., Yang, G., & Zheng, W. (2012). Job failures in high performance computing systems: A large-scale empirical study. *Computers and Mathematics with Applications*, 63(2), 365-377. <https://doi.org/10.1016/j.camwa.2011.07.040>
- Zhu, J., He, S., Liu, P., He, Q., Xu, Y., & Lyu, M. R. (2023). Loghub: A large collection of system log datasets for AI-driven log analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)* (pp. 355-366).
<https://doi.org/10.1109/ISSRE59848.2023.00071>
- Zou, J., & Petrosian, O. (2020). *Explainable AI: Using Shapely value to explain complex anomaly detection ML-based systems*. ResearchGate.
https://www.researchgate.net/figure/DeepLog-architecture_fig1_347324639